José Augusto Agripino de Oliveira

Análise de modelos de processamento de linguagem natural para classificação de notícias da UFRN

Natal – RN

Janeiro de 2025

José Augusto Agripino de Oliveira

Análise de modelos de processamento de linguagem natural para classificação de notícias da UFRN

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Prof. Dr. Ivanovitch Medeiros Dantas da Silva

Universidade Federal do Rio Grande do Norte – UFRN

Departamento de Engenharia de Computação e Automação – DCA

Curso de Engenharia de Computação

Natal – RN

Janeiro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Oliveira, Jose Augusto Agripino de.
Análise de modelos de processamento de linguagem natural para classificação de notícias da UFRN / Jose Augusto Agripino de Oliveira. - 2025.
61f.: il.

Monografia (Graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Engenharia de Computação, Natal, 2025.
Orientação: Dr. Ivanovitch Medeiros Dantas da Silva.

1. XGBoost - Monografia. 2. K-Means - Monografia. 3. Redução de Dimensionalidade - Monografia. 4. Processamento de Linguagem Natural - Monografia. 5. Classificação de Texto - Monografia. I. Silva, Ivanovitch Medeiros Dantas da. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

José Augusto Agripino de Oliveira

Análise de modelos de processamento de linguagem natural para classificação de notícias da UFRN

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Prof. Dr. Ivanovitch Medeiros Dantas da Silva

Trabalho aprovado. Natal – RN, 17 de Janeiro de 2025:

Prof. Dr. Ivanovitch Medeiros Dantas da Silva - Orientador
UFRN

Prof. Dr. Carlos Manuel Dias Viegas - Convidado
UFRN

Dr. Ignacio Sánchez Gendriz - Convidado
Posdoc/UFRN

Natal – RN
Janeiro de 2025

Dedico este trabalho aos meus pais, Nazarete e Agostinho, e aos meus avós, Raimunda e Geraldo, por toda paciência, amor e incentivo.

AGRADECIMENTOS

Aos meus pais, Francisca Nazarete Agripino de Oliveira e Agostinho Agripino de Oliveira Neto, por toda a dedicação e luta para que eu pudesse ter uma educação de qualidade.

Aos meus avós maternos, Raimunda Ribeiro de Oliveira e Geraldo José Damacena, por terem me criado com tanto amor, dedicando seus últimos anos de vida, e me incentivando a sempre seguir meus sonhos.

Aos meus avós paternos, Benedita Ribeiro e José Agripino, por toda a ajuda e cuidado.

À minha namorada, companheira e amiga Yannye Thaynnara Fernandes de Oliveira, por todo apoio, amor, carinho e por estar presente nos momentos mais difíceis.

Às tias que ajudaram na minha criação, Maria Raimunda e Francisca Raimunda, por todo o cuidado e preocupação durante tantos anos.

Às minhas tias, em especial Maria Socorro, Maria Margarida, Benedita Ribeiro, Raimunda Ribeiro e Maria Luiza, por todo o incentivo e ajuda.

Aos meus tios, em especial Raimundo Geraldo e Raimundo Nonato, por todos os momentos e exemplos dados.

Aos meus amigos da graduação em Ciências e Tecnologia, em especial Alicia Luzia, Aline Maria, Renan Bezerra e Joana Laabs, por toda a ajuda e pelos momentos de leveza que tornaram a minha jornada acadêmica mais leve.

Ao meu orientador e professor, Ivanovitch Medeiros Dantas da Silva, pelos ensinamentos, paciência, bom humor e oportunidades concedidas. Sua didática excepcional e visão do mundo me fizeram crescer como pessoa e como aluno, contribuindo grandemente para a minha formação acadêmica e profissional.

Ao professor Luiz Affonso Henderson Guedes de Oliveira, pela ideia deste trabalho e por toda a ajuda.

Aos meus primos, em especial Bruna Ribeiro, Rayssa Beatriz, Gislayne Oliveira, Vitor Alexandre e Randerson Pinheiro, por estarem presentes em todos os momentos importantes.

A todos que contribuíram direta e indiretamente com a minha formação.

"Feliz o homem que encontrou a sabedoria e alcançou o entendimento,
porque a sabedoria vale mais do que a prata,
e dá mais lucro que o ouro."
(Bíblia Sagrada, Provérbios 3, 13-14)

RESUMO

A Inteligência Artificial (IA) está transformando diversos setores, remodelando diariamente o que entendemos por “tecnologia”. Sua aplicação na análise da linguagem natural é de suma importância, com essa área se desenvolvendo exponencialmente todos os dias. Neste contexto de desenvolvimento,, este trabalho tem por objetivo analisar e comparar diferentes abordagens para o Processamento de Linguagem Natural (PLN) e classificação de texto aplicadas às notícias encontradas no site da Universidade Federal do Rio Grande do Norte (UFRN), classificando-as entre os temas de “ciências”, “eventos”, “vagas” e “informes”, além de disponibilizar interfaces com os melhores modelos. A metodologia adotada abrange desde a escolha das tecnologias até a implementação das *interfaces*. Nesse sentido, o *Data Version Control* (DVC) foi utilizado para o armazenamento e versionamento dos dados no desenvolvimento dos modelos, o GitHub para o versionamento de código, o Weight and Biases para versionamento e obtenção de gráficos relacionados ao treinamento dos modelos, além dos dados utilizados em conjunto com as *interfaces*, a API da *OpenAI* para realizar os *embeddings* dos textos, e o *Gradio*, em conjunto com o *Hugging Face*, para construir e disponibilizar os modelos. Por fim, os resultados obtidos demonstram que a maioria dos modelos apresentam uma alta acurácia, entretanto, aquele que utiliza o algoritmo de *XGBoost* apresentou um melhor resultado, conseguindo classificar, com precisão, os textos em linguagem natural. Visto que a IA está em constante crescimento e surgem cada vez mais abordagens para um certo fim, a discussão sobre qual é a melhor, a partir de um dado contexto, é imprescindível.

Palavras-chaves: *XGBoost*; *K-Means*; Redução de Dimensionalidade; Processamento de Linguagem Natural; Classificação de Texto.

ABSTRACT

Artificial Intelligence (AI) is transforming various sectors, reshaping what we understand as “technology” on a daily basis. Its application in natural language analysis is of utmost importance, with this field evolving exponentially every day. In this developmental context, this work aims to analyze and compare different approaches to Natural Language Processing (NLP) and text classification applied to the news found on the website of the Federal University of Rio Grande do Norte (UFRN). The classification is performed among the themes of “sciences”, “events”, “job openings” and “announcements”, while also providing interfaces with the best models. The adopted methodology encompasses everything from the choice of technologies to the implementation of interfaces. In this sense, Data Version Control (DVC) was used for data storage and versioning during the development of the models, GitHub for code versioning, Weight and Biases for versioning and generating graphs related to model training, the data used alongside the interfaces, OpenAI’s API for embedding the texts, and Gradio, in conjunction with Hugging Face, to build and deploy the models. Finally, the results show that most models achieved high accuracy; however, the one utilizing the XGBoost algorithm delivered the best performance, successfully classifying natural language texts with precision. Considering that AI is continuously growing and new approaches are constantly emerging, the discussion on which is the best approach for a given context is indispensable.

Keywords: XGBoost; K-Means; Dimensionality Reduction; Natural Language Processing; Text Classification.

LISTA DE ILUSTRAÇÕES

Figura 1 – Aproximação da técnica *boosting* no *ensemble learning*. 19
Figura 2 – Visão geral da abordagem proposta. 23
Figura 3 – Visão geral do código desenvolvido em Python. 24
Figura 4 – Visão da abordagem proposta para o modelo K-Means. 25
Figura 5 – Visão do algoritmo para o modelo K-Means com redução de dimensionalidade. 27
Figura 6 – Visão geral do código desenvolvido para o modelo XGBoost. 28
Figura 7 – Visão do código desenvolvido para o modelo XGBoost inicial. 29
Figura 8 – Visão do código desenvolvido para o modelo XGBoost final. 31
Figura 9 – Visualização do ambiente de intereção no *Hugging Face*. 33
Figura 10 – Fluxo para estruturação e interação com o modelo *K-Means*. 34
Figura 11 – Fluxo para interação com o modelo *K-Means* que faz uso do PCA. . . . 35
Figura 12 – Fluxo para estruturação e interação com o modelo *XGBoost*. 36
Figura 13 – Distribuição por tema das notícias classificadas manualmente. 38
Figura 14 – Nuvem de palavras dos dados utilizados no modelo *K-Means*. 39
Figura 15 – Frequência da quantidade de *tokens* por notícia. 40
Figura 16 – Matriz de confusão do modelo *K-Means*. 41
Figura 17 – Relatório de classificação para o modelo *K-Means*. 42
Figura 18 – Matriz de confusão do modelo *K-Means* com redução de dimensionaliade. 44
Figura 19 – Relatório de classificação para o modelo *K-Means* com PCA. 45
Figura 20 – Visualização dos *clusters* identificados usando PCA. 46
Figura 21 – Nuvem de palavras dos dados utilizados no modelo XGBoost inicial. . . 47
Figura 22 – Matriz de confusão do modelo XGBoost inicial. 48
Figura 23 – Relatório de classificação para o modelo XGBoost inicial. 49
Figura 24 – Nuvem de palavras dos dados utilizados no modelo XGBoost final. . . 50
Figura 25 – Matriz de confusão do modelo XGBoost final para dados de teste. . . . 51
Figura 26 – Relatório de classificação para o modelo XGBoost final para dados de teste. 52
Figura 27 – Matriz de confusão do modelo XGBoost final para notícias classificadas manualmente. 53
Figura 28 – Relatório de classificação do modelo XGBoost final para notícias classificadas manualmente. 54
Figura 29 – Relatório de classificação do modelo XGBoost final para notícias classificadas manualmente. 56

LISTA DE TABELAS

Tabela 1 – Hiperpâmetros para o modelo K-means. 40
Tabela 2 – Hiperpâmetros para a instância do PCA. 43
Tabela 3 – Hiperpâmetros para o modelo K-means com redução de dimensionalidade. 44
Tabela 4 – Hiperpâmetros para o modelo XGBoost inicial. 47
Tabela 5 – Hiperpâmetros para o modelo XGBoost final. 50
Tabela 6 – Emissões de carbono para os modelos propostos. 54
Tabela 7 – Comparação de tempos para classificar notícias usando diferentes algoritmos. 57

LISTA DE ABREVIATURAS E SIGLAS

IA	Inteligência Artificial
PLN	Processamento de Linguagem Natural
UFRN	Universidade Federal do Rio Grande do Norte
PCA	*Principal Component Analysis*
CSV	*Comma-separated values*
DVC	*Data Version Control*
MLOps	*Machine Learning Operations*
API	*Application Programming Interface*
W&B	*Weights & Biases*
HTML	*HyperText Markup Language*
ASCII	*American Standard Code for Information Interchange*
GCP	*Google Cloud Plataform*

SUMÁRIO

1 INTRODUÇÃO . . . 13
1.1 Contextualização . . . 13
1.2 Objetivos . . . 13
1.2.1 Geral . . . 13
1.2.2 Específicos . . . 14
1.3 Classificação da Pesquisa . . . 14
1.4 Motivação . . . 14
1.5 Estrutura do Trabalho . . . 15

2 FUNDAMENTAÇÃO TEÓRICA . . . 16
2.1 Processamento de Linguagem Natural . . . 16
2.1.1 Tokenização . . . 16
2.1.2 Lematização . . . 16
2.1.3 *Embeddings* . . . 16
2.1.4 Matriz TF-IDF . . . 17
2.1.5 Similaridade de cosseno . . . 18
2.1.6 *XGBoost* . . . 18
2.1.7 *K-Means* . . . 19
2.1.8 Redução de dimensionalidade: *Principal Component Analysis* (PCA) . . . 20
2.2 Ferramentas de Desenvolvimento e Monitoramento . . . 21
2.2.1 *OpenAI* . . . 21
2.2.2 *Gradio* . . . 21
2.2.3 *Data Version Control* (DVC) . . . 21
2.2.4 *Weights & Biases* (W&B) . . . 21
2.2.5 *Hugging Face* . . . 21

3 ABORDAGEM PROPOSTA . . . 23
3.1 Código dos modelos em Python . . . 24
3.1.1 K-Means . . . 25
3.1.2 K-Means com Redução de Dimensionalidade . . . 27
3.1.3 XGBoost . . . 28
3.1.3.1 Modelo Inicial . . . 29
3.1.3.2 Modelo Final . . . 31
3.2 Interação com os Modelos . . . 32
3.2.1 K-Means . . . 32

3.2.2 K-Means com redução de dimensionalidade 35
3.2.3 XGBoost 35

4 RESULTADOS 37
4.1 Modelos propostos 37
4.1.1 K-Means 38
4.1.2 *K-Means* com redução de dimensionalidade 43
4.1.3 XGBoost 46
4.1.3.1 Modelo inicial 46
4.1.3.2 Modelo final 49
4.1.4 Emissões de Carbono 53
4.1.5 Comparação dos Modelos 55
4.2 *Interfaces* do Usuário 55

5 CONCLUSÃO 58

REFERÊNCIAS 59

1 INTRODUÇÃO

Neste capítulo, busca-se contextualizar a crescente presença da Inteligência Artificial (IA) e suas implicações no cotidiano da sociedade contemporânea. Em particular, desca-se o uso da IA no Processamento de Linguage Natural (PLN). Além disso, apresenta-se a motivação deste tabalho, os objetivos, metodologia e a estrutura desta pesquisa.

1.1 Contextualização

A IA é o domínio da Ciência e Engenharia preocupado com o desenvolvimento de sistemas que permitem que computadores simulem a capacidade de pensamento lógico e inteligência humana (TECUCI, 2012). Nesse contexto, a classificação textual é de importância fundamental, e vem se destacando graças aos desenvolvimentos nos campos de mineração de texto e PLN. Os métodos de classificação possuem o objetivo comum de designar um certo rótulo para um determinado dado de entrada (GASPARETTO et al., 2022).

A IA passou por avanços significativos nos últimos anos, na pesquisa teórica e também nas aplicações práticas. Isso se dá pelo aumento do poder computacional, pelo grande volume de dados disponíveis e pelas melhorias na metodologia. Combinados, esses fatores têm viabilizado soluções robustas em várias áreas (SAMEK; MÜLLER, 2019).

Consoante Gasparetto et al. (2022), com a grande quantidade de dados textuais sendo criadas continuamente, há muito tempo superou as soluções manuais para classificação, isso significa que a classificação textual associada à IA não é apenas conveniente, mas também estritamente necessária.

1.2 Objetivos

1.2.1 Geral

Este trabalho tem por objetivo geral treinar, avaliar e comparar três diferentes modelos de classificação de texto com base nas notícias da Universidade Federal do Rio Grande do Norte (UFRN), além de disponibilizar *interfaces* interativas para utilizá-los. Esses modelos irão classificar textos entre os temas de “eventos”, “ciências”, “informes” e “vagas”.

1.2.2 Específicos

Para alcançar o prósito citado anteriormente, este trabalho se orienta pelos objetivos específicos a seguir:

- Classificar um número razoável de notícias entre os temas propostos para servir como base para desenvolver um dos modelos citados, e para calcular a acurácia de cada modelo treinado.
- Aplicar técnicas de processamento de linguagem natural e de classificação de texto para desenvolver os objetos de estudo citados.
- Avaliar os modelos de forma isolada, expondo métricas relevantes para análise.
- Comparar os modelos e observar qual apresentou o melhor resultado.
- Disponibilizar três ambientes interativos, utilizando técnicas de *Machine Learning Operations* (MLOps), para que o usuário possa classificar textos.

1.3 Classificação da Pesquisa

Para alcançar os objetivos deste trabalho, foi necessário definir a classificação quanto ao método de pesquisa. Nesse contexto, no que diz respeito à natureza da pesquisa, ela se enquadra como aplicada, uma vez que busca produzir conhecimentos aplicáveis para solucionar um problema específico: o processamento e classficação de textos (WAZLAWICK, 2009) e também como comparativa, já que visa analisar e interpretar os paralelos entre os objetos de estudo (SILVA, 2016).

Em relação à abordagem dos dados, a pesquisa foi categorizada como quantitativa, tendo-se em vista as métricas utilizadas para medição da eficiência.

1.4 Motivação

Existe um volume abundante de texto em linguagem natural no mundo conectado, embora intrínseco a esse volume de dados haja grande conhecimento a ser explorado, está se tornando cada vez mais difícil de disseminá-lo, especialmente dado um tempo de vida humana. O PLN visa fazer esse trabalho com eficiência e precisão, como um humano faz (CHOWDHARY; CHOWDHARY, 2020).

Tendo-se em vista essa grande quantidade de documentos textuais diários que é necessário lidar, para Ikonomakis et al. (2005), a classificação automática de textos não é apenas importante, mas sim uma necessidade.

Com o *K-Means* sendo o método de agrupamento não supervisionado mais conhecido e usado (SINAGA; YANG, 2020), e a alta acurácia e robustez do XGBoost (LI et al., 2020), esses foram os algoritmos escolhidos para as tarefas de classificação das notícias da Universidade Federal do Rio Grande do Norte (UFRN).

Portanto, o trabalho visa observar a eficiência e precisão dos modelos desenvolvidos para a classificação das notícias da UFRN, apresentando três modelos: *K-Means*, *K-Means* com redução de dimensionalidade - para duas dimensões com o intuito de visualizar os grupos resultantes - e *XGBoost*.

1.5 Estrutura do Trabalho

Este trabalho apresenta uma introdução sobre o tema, mostrando os fatores que motivam a implantação da ideia, além da classificação e dos objetivos da pesquisa. Em sequência, o Capítulo 2 aborda o referencial teórico utilizado. O Capítulo 3, por sua vez, explica a abordagem proposta para o desenvolvimento e implantação dos modelos, enquanto o Capítulo 4 trata dos resultados obtidos. Por fim, o Capítulo 5 traz as principais conclusões e contribuições deste trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Processamento de Linguagem Natural

Segundo Brownlee (2017), o estudo sobre Processamento de Linguagem Natural, com sigla PLN, tem origem há mais de 50 anos, e é definida como sendo a manipulação automática da linguagem natural, como fala e texto, por um software. Suas raízes são provenientes da relação da liguística tradicional com a evolução computacional. Desse modo, o principal objetivo do PLN é implementar sistemas capazes de interagir com a linguagem humana, seja ela por texto ou por voz.

A linguística computacional é o estudo da língua usando as ferramentas da ciência da computação. A linguística clássica pode ser a linguística computacional de hoje, já que abrange métodos tradicionais quanto abordagens modernas (BROWNLEE, 2017).

Logo, o PLN consiste na manipulação computacional na linguagem natural para compreender declarações humandas com o intuito de dar respostas úteis, conforme Bird, Klein e Loper (2009).

2.1.1 Tokenização

A tokenização é uma das muitas etapas possíveis de pré-processamento que podem ser aplicadas aos textos que vão interagir com um modelo de *machine learning*. Para Grefenstette (1999), a tokenização significa dividir o texto de entrada, que para o computador é apenas uma longa string, em subunidades, chamadas tokens.

2.1.2 Lematização

A lematização, segundo Plisson et al. (2004), é o processo de encontrar a forma normalizada de uma palavra. É um importante passo para o PLN e muitos outros campos que lidam com a linguística no geral. Essa abordagem também permite gerar palavras chaves para ferramentas de busca, entre outras aplicações. Consiste em retirar ou adicionar um sufixo a uma palavra para obter sua forma canônica, recorrendo ao infinitivo dos verbos e ao singular dos substantivos e adjetivos.

2.1.3 *Embeddings*

Para o Processamento de Linguagem Natural, os *embeddings* vem se mostrando como uma ferramenta imprescindível para manipulação dos dados. Como uma definição

geral, *embeddings* são dados transformados em uma matriz de dimensão n, onde n é um número inteiro e maior que zero (BOYKIS, 2023).

Com a publicação do artigo *Word2Vec* pelo *Google*, houve um marco na utilização dos *embeddings* para gerar representações numéricas dos dados. Todavia, foi com a arquitetura *Transformer* que os *embeddings* tiveram as capacidades e relevância amplamente aumentadas. Com ela, foi possível melhorar a compreensão e representação de contextos nos dados textuais, reforçando, assim, os *embeddings* no PLN (BOYKIS, 2023).

Com isso, um *embedding* é, segundo Boykis (2023), um vetor multidimensional que encapsula a semântica e nuâncias textuais do dado de entrada. Por seu desempenho e versatilidade, *embeddings* têm ocupado um lugar importante no PLN, sendo utilizados em diversas tarefas, desde sistemas de recomendação até classificação textual.

2.1.4 Matriz TF-IDF

Em grandes conjuntos de textos, algumas palavras estarão muito presentes, como "a", "o", "e", "é", entre outras. Elas carregam, assim, pouca informação sobre o real conteúdo do documento. Com a pura contagem das palavras, os termos citados que aparecem em muitos documentos iriam ofuscar aqueles que são raros, mas carregam um alto valor semântico para o texto (DEVELOPERS, 2024).

TF-IDF é a sigla para *Term Frequency - Inverse Data Frequency*. A frequência do termo (TF) representa a frequência com que uma palavra (termo) aparece em um documento em relação ao número total de palavras do documento que está sendo analisado. A Equação 2.1 mostra como calcular a TF de uma palavra (TRIPATHI, 2018).

$$TF_{i,j} = \frac{f_{i,j}}{N_j} \tag{2.1}$$

Onde:

- $f_{i,j}$ é o número de vezes que a palavra t_i aparece no documento d_j.
- N_j é o número total de termos no documento d_j.

Segundo Tripathi (2018), o inverso da frequência dos dados (IDF) mede a importância de uma palavra em todo o conjunto de documentos. O objetivo por trás do IDF é dar uma maior importância para os termos mais raros em detrimento dos comuns. Na Equação 2.2 é mostrada a fórmula para calculá-lo.

$$IDF_i = \log\left(\frac{N}{df_i}\right) \tag{2.2}$$

Em que:

- N é o número total de documentos.
- df_i é o número de documentos em que o termo t_i aparece.

Desse modo, para calcular o TF-IDF da palabra i, que pertece ao documento j, multiplica-se a Equação 2.1 pela Equação 2.2, como mostra a Equação 2.3

$$\text{TF-IDF}_{i,j} = TF_{i,j} \times IDF_i \tag{2.3}$$

.

O resultado é uma matriz esparsa, ou seja, que possui várias posições iguais a zero, uma vez que existem muitos termos que não estarão em um determinado documento textual, o que gera um TF-IDF = 0.

2.1.5 Similaridade de cosseno

A similaridade de cosseno é um componente básico para várias aplicações de mineração de textos, recuperação de informações, PLN, entre outras utilizações. Nesse contexto, é utilizada para avaliar o quanto dois segmentos de textos são similares, mensurando o ângulo entre dois vetores que representam os textos a serem comparados. Isso é feito ao calcular o produto escalar entre os dois vetores normalizados (LI; HAN, 2013).

Conforme Li e Han (2013), entre várias métricas que buscam medir a semelhança entre dois textos, a similaridade de cosseno é a mais famosa. A Equação 2.4 mostra como calcular esta métrica dado dois vetores $\vec{v}$ e $\vec{w}$ de dimensão N.

$$\text{Cosseno}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{\|\vec{v}\|\|\vec{w}\|} = \frac{\sum_{i=1}^{N} v_i \times w_i}{\sqrt{\sum_{i=1}^{N} v_i^2} \times \sqrt{\sum_{i=1}^{N} w_i^2}} \tag{2.4}$$

.

2.1.6 *XGBoost*

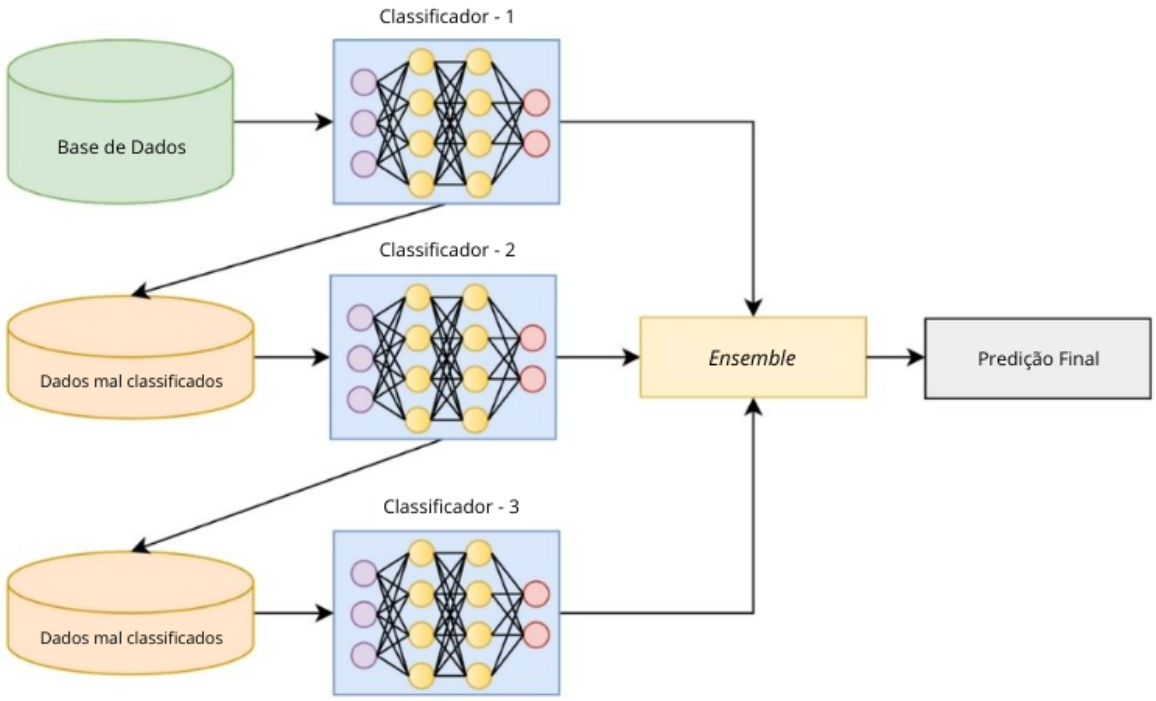
Para entender o algoritmo *XGBoost*, é preciso entender um conceito que rege o seu desenvolvimento, o de comitê de máquinas, também conhecido como *ensemble learning*.

Um comitê de máquinas consiste em priorizar os pontos fortes de vários modelos individuais para construis um modelo preditivo mais robusto e preciso. Com isso, as principais técnicas de *ensemble learning* são *bagging*, *boosting* e *stacking* (BOESCH, 2024).

O *XGBoost* utiliza a técnica de *boosting*, como o próprio nome evidencia. O *boosting* é um método sequencial de comitê de máquina em que vários modelos simples são construídos

um após o outro. Cada novo modelo foca em corrigir os erros do modelo anterior, atribuindo pesos maiores aos pontos de dados classificados de forma incorreta. Esses pesos e dados são utilizados na interação seguinte para ajustar o modelo da melhor maneira. A predição final vem da combinação da previsão de todos os modelos individuais, aplicando-se um método de votação (BOESCH, 2024). A Figura 1 ilustra como a técnica citada funciona para gerar a predição final.

Figura 1 – Aproximação da técnica *boosting* no *ensemble learning*.



Fonte: Adaptada de Boesch (2024).

Consoante Li et al. (2020), o *XGBoost* vem ganhando popularidade devido à sua acurácia e robustez. O *XGBoost* utiliza o método *boosting*. Ele combina árvores de classificação e regreção - *classification and regression trees* (CART), do inglês -, mas redefine a função objetivo considerando tanto a perda de treinamento, mais conhecida por seu termo em inglês *training loss*, quanto a complexidade das árvores para diminuir a chance de sobreajuste, ou *overfitting*, do modelo.

2.1.7 *K-Means*

O *K-Means* é, geralmente, o método de agrupamento mais conhecido e usado. Ele é um método de aprendizado não supervisionado para reconhecer e agrupar padrões, entretanto, precisa ser definido o número de agrupamentos, ou *clusters*, que os dados possuem. É desse conceito que vem o "K" em *K-Means*. Existem várias aborgadens para encontrar uma quantidade ótima de *clusters*. Todavia, neste trabalho, é sabido que a

quantidade de *clusters* são quatro, já que as notícias se dividem em quatro temas (SINAGA; YANG, 2020).

A divisão dos dados em *clusters* é um método para encontrar estruturas de grupos em uma base de dados. É caracterizado por agrupar dados com grande similaridade em um mesmo *cluster*, e separar em diferentes *clusters* aqueles com grande diferença (SINAGA; YANG, 2020).

$$J = \sum_{j=1}^{k} \sum_{i=1}^{n} \|x_i^{(j)} - c_j\|^2 \tag{2.5}$$

Segundo Bishop (2006), a Equação 2.5 mostra como calcular quais amostras pertencem a cada cluster. O intuito é minimizar a função objetivo J, de forma que a distância ao quadrado das amostras x_i em relação ao centróide c_j do *cluster* j seja a menor possível. Os centróides, inicialmente, podem ser escolhidos de forma aleatória ou não, porém, eles são otimizados para que os valores ótimos sejam encontrados. Isso significa que, ao final da execução do algortimo, os pontos de dados estarão agrupados de forma que cada um esteja o mais próximo possível de seu respectivo centróide.

2.1.8 Redução de dimensionalidade: *Principal Component Analysis* (PCA)

A redução de dimensionalidade pode ser feita de duas formas diferentes: apenas mantendo as variáveis mais relevantes, sem alterá-las, do conjunto de dados original (essa técnica é chamada de *feature selection*), ou reduzindo o número de variáveis no conjunto de dados transformando as variáveis originais em um novo conjunto, que é uma combinação dos dados originais (essa técnica é chamada de *dimensionality reduction*). Neste trabalho é utilizada a segunda técnica (SORZANO; VARGAS; MONTANO, 2014).

Segundo Sorzano, Vargas e Montano (2014), a redução de dimensionalidade não é nova na estatística. Na verdade, o *Principal Component Analysis* (PCA) data de 1901, desenvolvido por Karl Pearson. Entretanto, conforme cita Kurita (2019), o PCA é uma ferramenta padrão na análise de dados moderna e é usada por quase todas as disciplinas científicas. A ideia central é encontrar um novo sistema de coordenadas que os dados possam ser representados com muito menos variáveis que possuem um erro significante (SORZANO; VARGAS; MONTANO, 2014).

O Objetivo do PCA é identificar a base, ou sistema de coordenadas, mais significativa para expressar, de maneira nova, um conjunto de dados. Espera-se que essa nova base revele a estrutura oculta do conjunto de dados e filtre o ruído (KURITA, 2019).

2.2 Ferramentas de Desenvolvimento e Monitoramento

2.2.1 *OpenAI*

A *OpenAI* é uma empresa de pesquisa e implantação de Inteligência Artificial (IA) (OPENAI, 2024a). Entre as técnicas e tecnologias desenvolvidas pela empresa, existem os modelos de embeddings. Dentre eles, este trabalho utiliza o "text-embedding-3-large", que é o modelo mais eficiente de embeddings da *OpenAI*, apresentando uma alta performance. Além disso, a empresa disponibiliza uma API para acesso ao modelo, o que facilita o desenvolvimento (OPENAI, 2024b).

2.2.2 *Gradio*

Grádio é um pacote Python de código aberto que permite a construção rápida de aplicações web para um modelo de *machine learning*, *Application Programming Interface* (API) ou uma função arbitrária em Python (GRADIO, 2024). Neste trabalho, o *Gradio* é utilizado para criar ambientes de intereção com os modelos propostos.

2.2.3 *Data Version Control* (DVC)

O *Data Version Control* (DVC) é um sistema de controle de versão de código aberto para projetos de ciência de dados e *machine learning*. É possível rastrear e controlar as versões dos grander conjuntos de dados utilizados para o treinamento dos modelos (DVC, 2024). Neste trabalho, é utilizado para versionar os dados para o desenvolvimento do modelo.

2.2.4 *Weights & Biases* (W&B)

O *Weights & Biases* (W&B) é uma plataforma de *machine learning* voltada para melhorar o desenvolvimento dos modelos. Ela oferece o rastreio dos experimentos, o versionamento e iteração rápida em conjunto de dados, avaliação do desempenho dos modelos, reprodução dos modelos e visualização dos resultados (BIASES, 2024).

Para os modelos propostos neste trabalho, estão sendo utilizadas as funcionalidades de rastreio dos experimentos, o versionamento dos dados em produção, dado o seu alto desempenho com grandes conjuntos de dados e o armazenamento dos modelos criados.

2.2.5 *Hugging Face*

O *Hugging Face* é uma empresa que estava focada inicialmente no desenvolvimento de *chatbots*. Porém, com o tempo, a empresa evoluiu para criar sua própria infraestrutura para o PLN, oferecendo uma série de bibliotecas que facilitam o uso e desenvolvimento de

modelos de PLN. A empresa disponibiliza modelos pré-treinados, conjuntos de dados para treinar modelos de IA e *spaces*, que são aplicações *web* de IA na estrutura do *Hugging Face* - produzidos por um usuário (PEREIRA, 2024).

Os *spaces* são utilizados neste trabalho para hospedar os ambientes de interação com o modelo desenvolvidos com o Gradio.

3 ABORDAGEM PROPOSTA

Neste capítulo, é apresentada a abordagem proposta para o treinamento de três modelos de classificação textual: dois fazem uso do algoritmo *K-Means*, com um deles utilizando a redução de dimensionalidade, e o terceiro modelo utilizando o algoritmo *XGBoost*. Ademais, é exposta a abordagem de desenvolvimento de *interfaces* interativas para comunicação entre o usuário e os modelos.

A Figura 2 representa uma visão de alto nível da abordagem desenvolvida neste trabalho. Inicialmente há o desenvolvimento dos três modelos de classificação de texto citados a partir da captura dos dados brutos e do seu preprocessamento, visto em (a) e detalhado na seção 3.1. Já em (b) é possível observar as ferramentas de suporte utilizadas no desenvolvimento e gestão do projeto, são elas o *Data Version Control* (DVC), a *Application Programming Interface* (API) da *OpenAI* e o *Weights & Biases* (W&B). O DVC armazena por meio do *Google Drive* e versiona os dados obtidos e processados na etapa de treinamento dos modelos. A API da *OpenAI* é utilizada para obter os *embeddings* dos textos, ou seja, a sua representação numérica na forma vetorial. O W&B é usado para armazenar os modelos, *logs*, imagens e métricas gerados ao executar os *scripts Python* desenvolvidos. Por fim, em (c), são mostradas as tecnologias utilizadas para desenvolver *interfaces* para interagir com os modelos treinados - mais detalhes na seção 3.2.

Figura 2 – Visão geral da solução proposta.

Fonte: O autor, 2024.

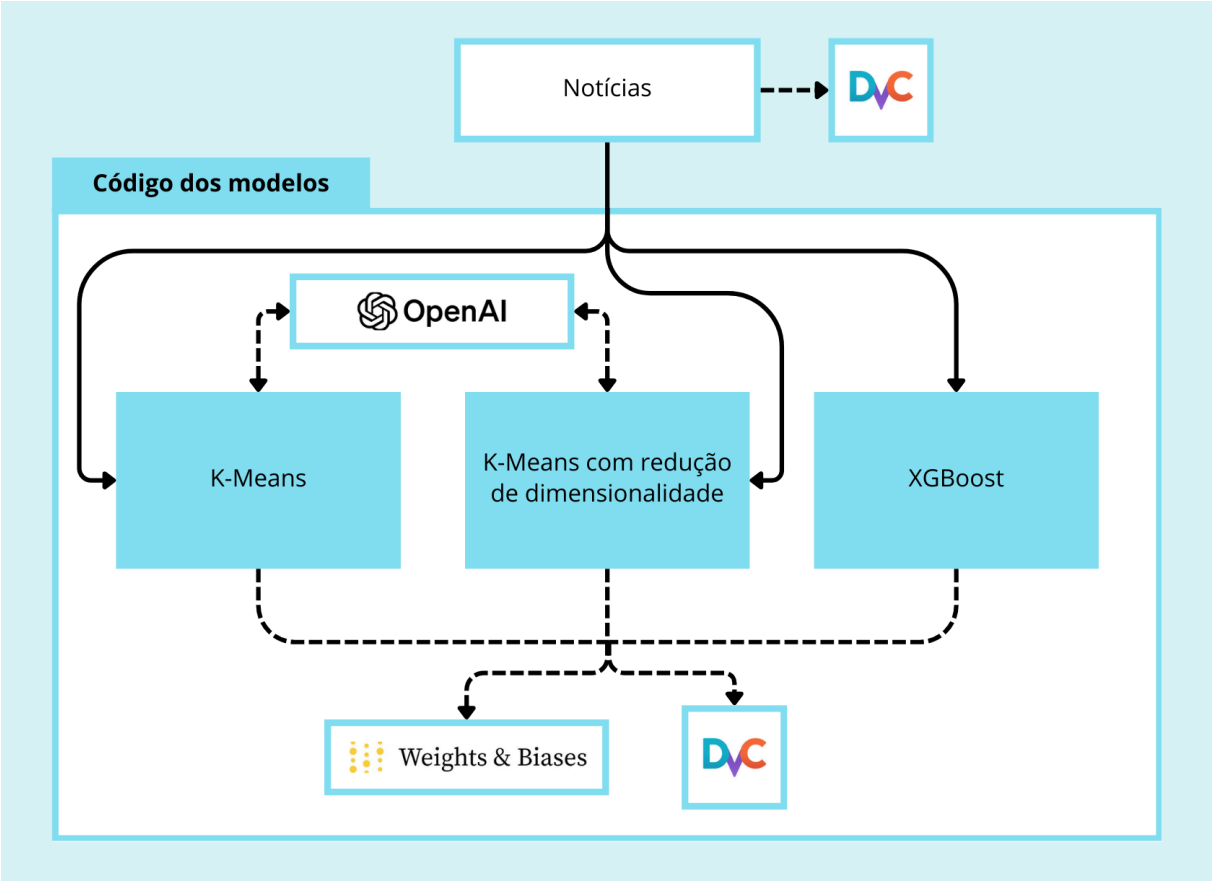
A seguir, serão explicados os procedimentos adotados em cada etapa. Uma vez que as ferramentas de desenvolvimento mostradas na Figura 2 estão atreladas aos processos (a) e (c), a forma como foram utilizadas será mostrada ao decorrer da explicação.

O código utilizado para o desenvolvimento dos modelos está disponível neste link: https://github.com/AugustoOliveira099/TCC.

3.1 Código dos modelos em Python

Os três modelos propostos foram implementados e treinados localmente, em Python. Inicialmente são capturadas as notícias, como são as mesmas para as abordagens propostas, esse processo é feito apenas uma vez, depois, cada implementação as preprocessa e tem o treinamento do modelo feito. Ao final, cada um deles é salvo no W&B e os dados rotulados são versionados e salvos por meio do DVC. É possível ver uma representação deste processo na Figura 3.

Figura 3 – Visão geral do código desenvolvido em Python.



Fonte: O autor, 2024.

Os textos utilizados para a realização deste trabalho são notícias da UFRN, obtidas a partir de uma API fornecida pela Superintendência de Tecnologia da Informação (STI) da UFRN. Os dados possuem apenas o título e o corpo da notícia, sendo essas informações versionadas e armazenadas por meio do DVC em conjunto com o *Google Drive*.

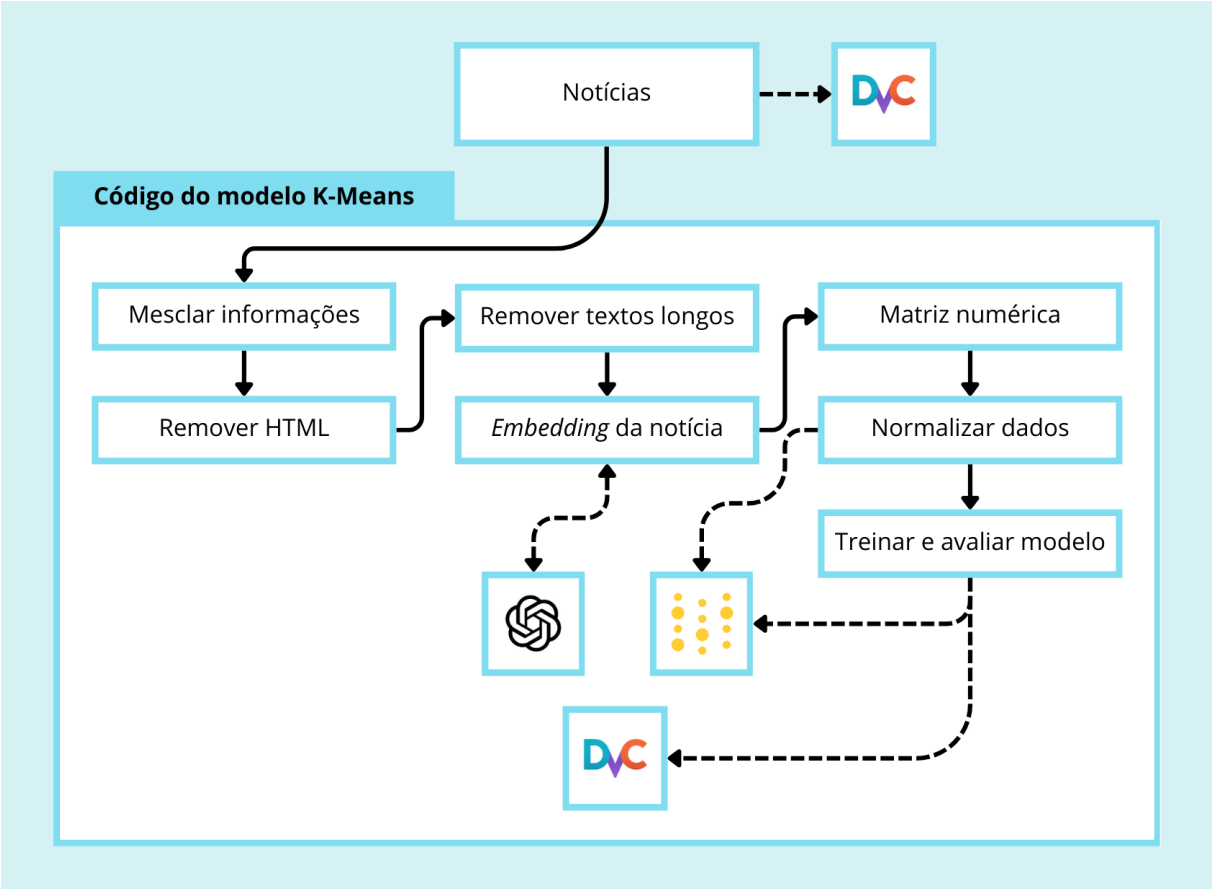
Parte dessas notícias foram classificadas manualmente com dois objetivos: utilizá-las para a técnica de aprendizado semi-supervisionado no modelo com o algoritmo *XGBoost* e para verificar o nível de confiabilidade dos classificadores. Dessa forma, os modelos com o algoritmo K-Means não utilizam essas notícias no treinamento. Já o modelo com XGBoost

as utiliza apenas no modelo inicial - conceito elucidado na subseção 3.1.3. Todos eles fazem uso delas na avaliação do modelo.

3.1.1 K-Means

Nesta abordagem de aprendizado não supervisionado, são realizados os passos mostrados na Figura 4:

Figura 4 – Visão da abordagem proposta para o modelo K-Means.



Fonte: O autor, 2024.

1. **Mesclar informações:** A primeira etapa consiste em unir as informações de título e corpo da notícia, uma vez que ambas as características são importantes para identificar o seu tema. Nesse sentido, o título é antecedido de "Título:", enquanto o corpo da requisição é antecedido de "Conteúdo".

2. **Remover *HyperText Markup Language* (HTML):** A API responsável por prover as notícias retorna o texto com *tags* HTML e com informações desnecessárias, como fotos e suas fontes. Esse conteúdo seria extremamente prejudicial para o modelo, atrapalhando nas predições a serem realizadas. Desse modo, é necessário retirá-lo.

3. **Remover textos longos:** Os textos serão convertidos em representações vetoriais por meio dos modelos de *embeddings* da *OpenAI*. Esses modelos aceitam uma quantidade máxima de tokens, a depender do modelo utilizado, neste trabalho será utilizado o modelo "text-embedding-3-large", que permite uma entrada com no máximo 8191 tokens. Com isso, o texto é convertido em uma representação por tokens e é removido caso ultrapasse o limite permitido. Para o conjunto de dados que está sendo usado, não houve remoção.

4. **Utilizar os modelos de *embeddings* da *OpenAI*:** Neste passo, é utilizado, de fato, o modelo de *embedding* "text-embedding-3-large", por meio de requisições para a API da *OpenAI* contendo os textos a serem processados. O resultado é versionado pelo DVC.

5. **Transformar a representação vetorial do texto em uma matriz numérica:** Os vetores obtidos na etapa anterior estão armazenados como strings, é necessário transformá-los em uma matriz.

6. **Normalizar dados:** Esse é um passo muito importante no preprocessamento dos dados para a utilização do modelo em questão, uma vez que ele utiliza a distância Euclidiana para agrupar os dados. Está sendo feita a normalização *Z-score*, que consiste em transformar os dados para que contenham média 0 e desvio padrão 1. O artefato responsável por normalizar a matriz contendo os *embeddings* é salvo no W&B para futura utilização.

7. **Treinar modelo:** O treinamento do modelo K-Means consiste em formar *clusters*, ou seja, grupos de dados agrupados de acordo com sua similaridade. Neste caso, a quantidade de clusters é a mesma quantidade de temas possíveis para as notícias: quatro. Após ajustar o modelo aos dados, é feita a previsão de agrupamento das notícias, como existem quatro possibilidades, são rotuladas de zero a três. Então, com esse procedimento concluído, é preciso saber qual tema cada rótulo representa. Para fazer essa tarefa, são escolhidas cinco notícias, de forma aleatória, de cada cluster, em seguida, essas notícias são formatadas como na subseção 3.1.1, item (1). A partir disso, elas são enviadas para o modelo de linguagem GPT-4 da *OpenAI*, por meio de sua API, e lhe é perguntado qual o tema das notícias, considerando as quatro possibilidades. Em função disso, é possível saber o tema que cada cluster representa. Em seguida, os dados classificados são armazenados no *Google Drive* por meio do DVC, assim como no W&B com o intuito de utilizá-los no ambiente de produção. O modelo também é salvo no W&B.

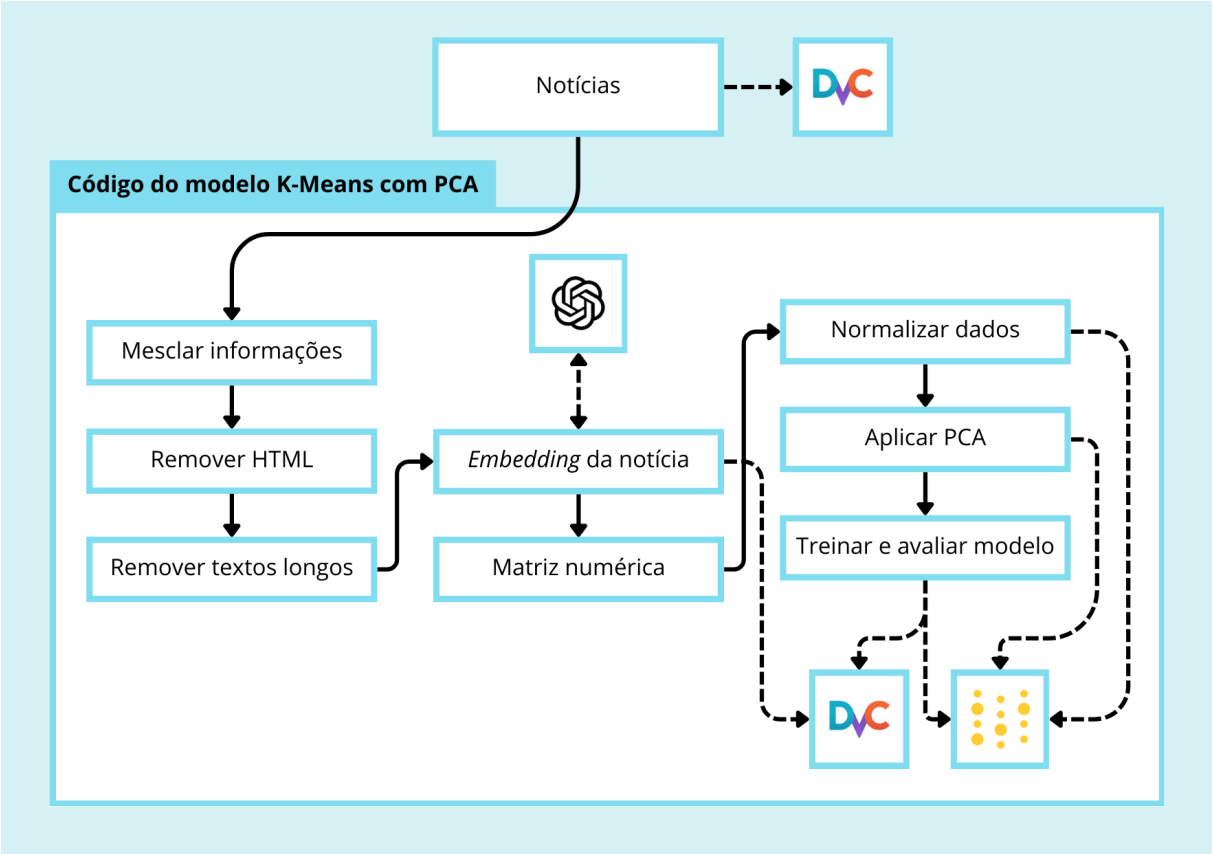
8. **Avaliar modelo:** Como dito, o conjunto de dados utilizado para esta etapa são as notícias classificadas manualmente, em razão da confiabilidade da *label*. As métricas

observadas foram acurácia, *recall*, precisão e *F1 score*, além da matriz de confusão. O resultado dessas métricas, assim como as plotagens relacionadas a elas, são armazenadas no W&B.

3.1.2 K-Means com Redução de Dimensionalidade

Para este modelo, todos os passos apresentados na subseção 3.1.1 são seguidos, assim, só é preciso capturar os embeddings das notícias uma vez, já que a base de dados é a mesma para ambos os modelos. A diferença entre eles é que neste é feito o uso da redução de dimensionalidade. Essa técnica foi adicionada para verificar o desempenho do modelo resultante, e para que seja possível visualizar os *clusters* formados. Na Figura 5 é mostrada a visão geral do algoritmo utilizado.

Figura 5 – Visão do algoritmo para o modelo K-Means com redução de dimensionalidade.



Fonte: O autor, 2024.

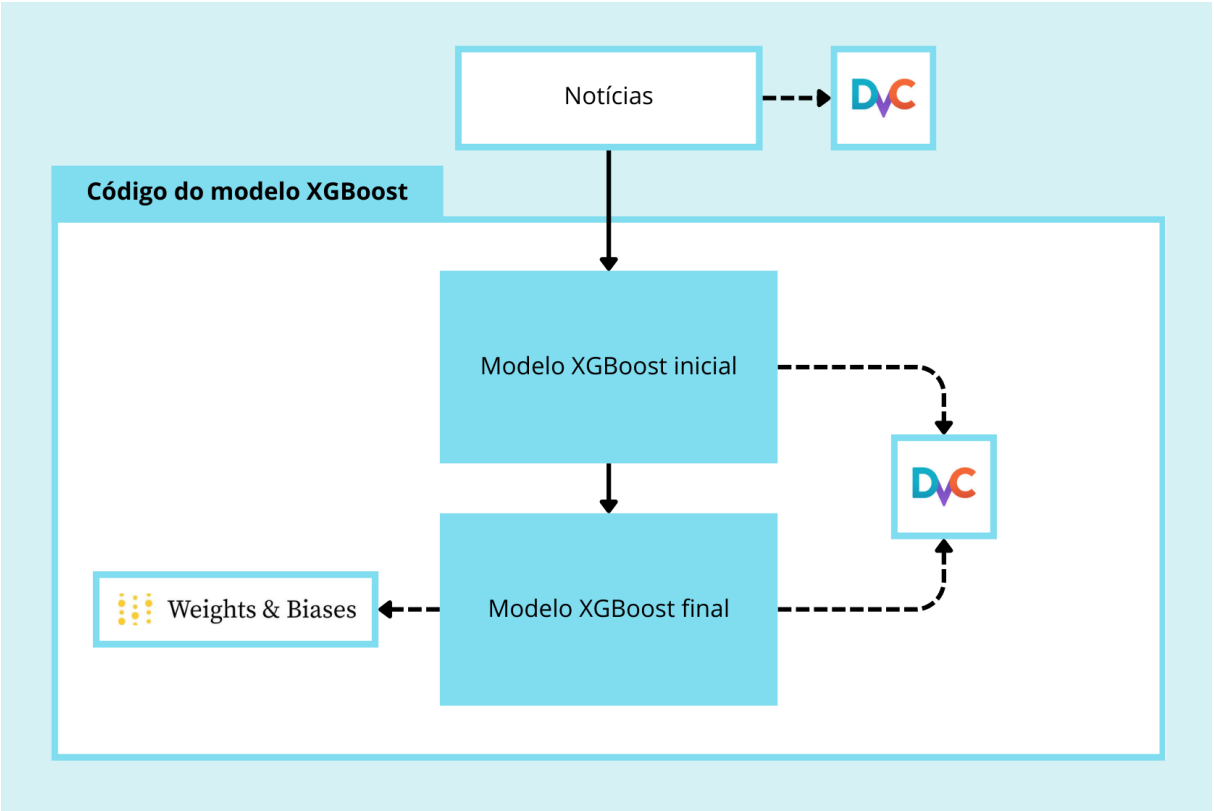
Nesse contexto, é feito uso da técnica de redução de dimensionalidade linear *Principal Component Analysis* (PCA) para representar os vetores numéricos em duas dimensões, ou seja, são reduzidos para dois componentes. PCA é um algoritmo linear que procura preservar a variação global dos dados. A instância desse algoritmo, depois de ajustada aos dados de treinamento, é salva no W&B, assim como os dados resultantes no formato

CSV, contendo uma coluna com os dois termos que buscam reproduzir as características do embedding completo, denominada "pca".

3.1.3 XGBoost

O modelo *XGBoost* desenvolvido utiliza a técnica de aprendizado semi-supervisionado, com a abordagem *Self-Training*, que consiste em treinar um modelo com um pequeno conjunto de dados, depois utilizá-lo para classificar um conjunto maior, capturando apenas os dados com uma boa probabilidade da previsão realizada pelo modelo estar correta - está sendo utilizado o limiar de 80% -, e, por fim, treinar um novo modelo com essa nova base de dados rotulada. Por consequência disso, o processo de construção do classificador em questão é dividido em duas fases: treinar um modelo inicial e um final, como é obervado na Figura 6.

Figura 6 – Visão geral do código desenvolvido para o modelo XGBoost.



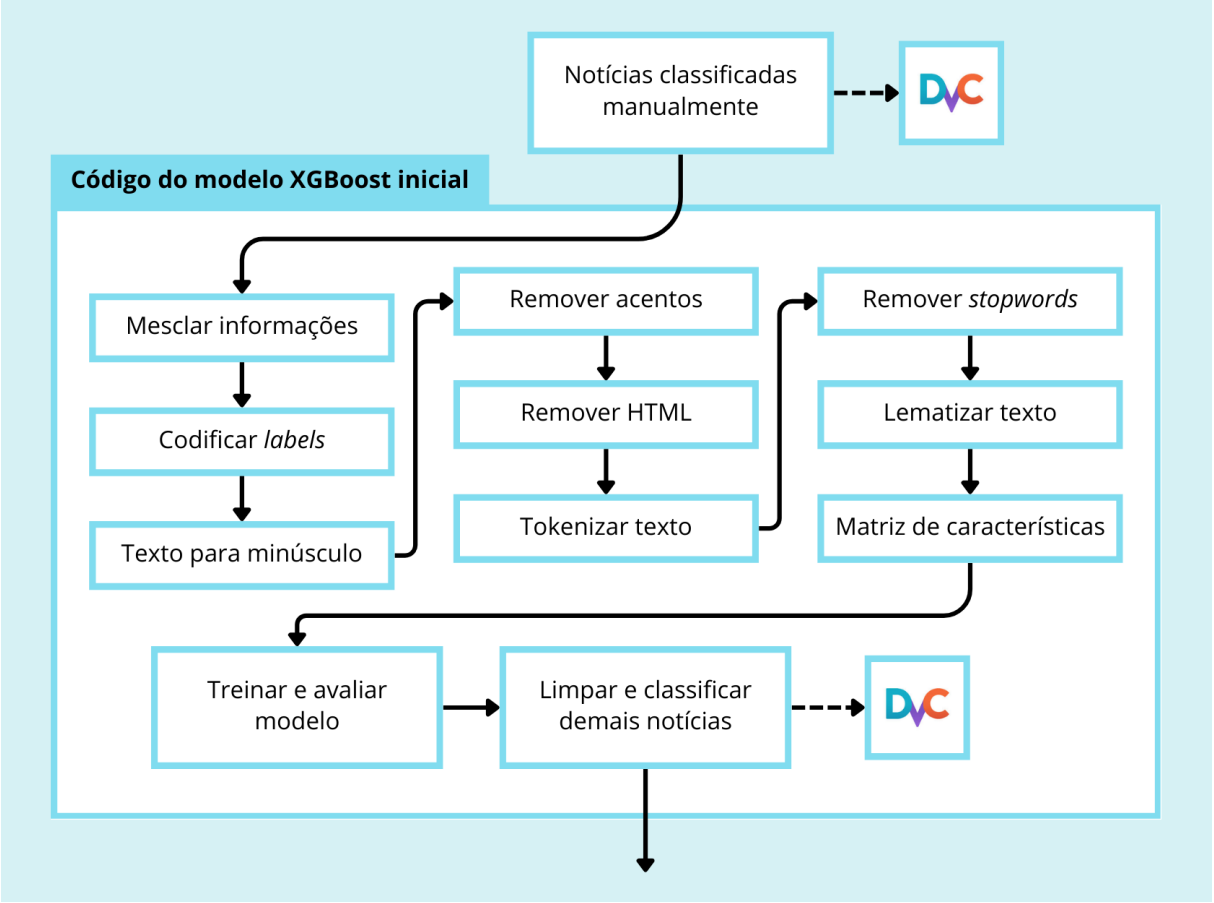
Fonte: O autor, 2024.

Nesta abordagem, além dos itens citados, as etapas de preprocessamento são diferentes das que existem nos modelos que fazem uso do algoritmo *K-Means*, uma vez que não é utiliza a API da *OpenAI* para obter a representação numérica do texto. Isso impacta diretamente no processo de limpeza e transformação das notícias. A seguir estão os processos realizados por cada um dos modelos que utilizam o *XGBoost*.

3.1.3.1 Modelo Inicial

A Figura 7 elucida os passos realizados para preprocessamento e treinamento do primeiro modelo. Nele não há a comunicação com o W&B, visto que é um modelo inicial e o interesse por trás de sua implementação é apenas o conjunto de dados rotulados resultante dela. Esse, por sua vez, é armazenado e versionado com o uso do DVC e do W&B, também visando a sua utilização no ambiente de produção.

Figura 7 – Visão do código desenvolvido para o modelo XGBoost inicial.



Fonte: O autor, 2024.

A seguir está cada um dos passos explicados.

1. **Combinar informações de título e corpo da notícia:** Este é um processo igual ao descrito na subseção 3.1.1, item (1).

2. **Codificar labels:** Os modelos não conseguem se ajustar a variáveis categóricas, logo, é preciso convertê-las em números. Para isso, a biblioteca *scikit-learn* fornece a classe *LabelEncoder*, que se ajusta aos dados e os codifica. Como os textos em questão são classificados entre "Ciências", "Eventos", "Vagas" e "Informes", é preciso aplicar o processo descrito. Desse modo, é feito o mapeamento de 0 a 3, para representar cada um dos temas.

3. **Converter o texto para minúsculo:** Para ter um bom resultado, é preciso padronizar os textos. Dito isso, converter o texto para minúsculo é essencial, uma vez que letras minúsculas e maiúsculas são interpretadas de forma diferente devido aos seus códigos na tabela *American Standard Code for Information Interchange* (ASCII) serem diferentes.

4. **Remover acentos:** Com a mesma premissa do item anterior, a remoção de acentos é primordial. As letras com e sem acento também são interpretadas de forma diferente, o que dificulta o seu processamento adequado.

5. **Remover HTML dos textos**: O processo realizado nesta etapa é igual ao descrito na subseção 3.1.1, item (2).

6. **Tokenizar texto:** A tokenização consiste em dividir o texto em palavras e pontuações, é importante para os passos subsequentes.

7. **Remover *stopwords*:** As *stopwords* são palavras comuns e geralmente irrelevantes que devem ser retiradas dos dados de treinamento, já que, em sua maioria, não agregam valor semântico ao texto. Com o texto tokenizado, é possível verificar e remover as palavras que se enquadram como *stopwords*.

8. **Lematização do texto:** A lemmatização consiste em encontrar a forma normalizada de uma palavra (PLISSON et al., 2004), ou seja, sem conjugação. Com isso, ao receber o texto, ainda tokenizado, cada palavra é posta em sua forma normal.

9. **Matriz de características:** Com os textos sem ruídos, é possível obter as suas representações numéricas, necessárias para o treinamento do modelo. Para isso, uma instância da classe *TfidfVectorizer*, que pertence à biblioteca *scikit-learn*, é utilizada. Após o ajuste do objeto aos dados e posterior transformação dos textos, uma matriz de características (TF-IDF) com a importância de cada palavra em uma notícia em relação ao conjunto de notícias é retornada.

10. **Treinar e avaliar modelo:** Com o texto processado, é possível treinar e avaliar o modelo XGBoost. As métricas observadas para avaliação foram acurácia, precisão, *recall* e *F1 score*, além da matriz de confusão.

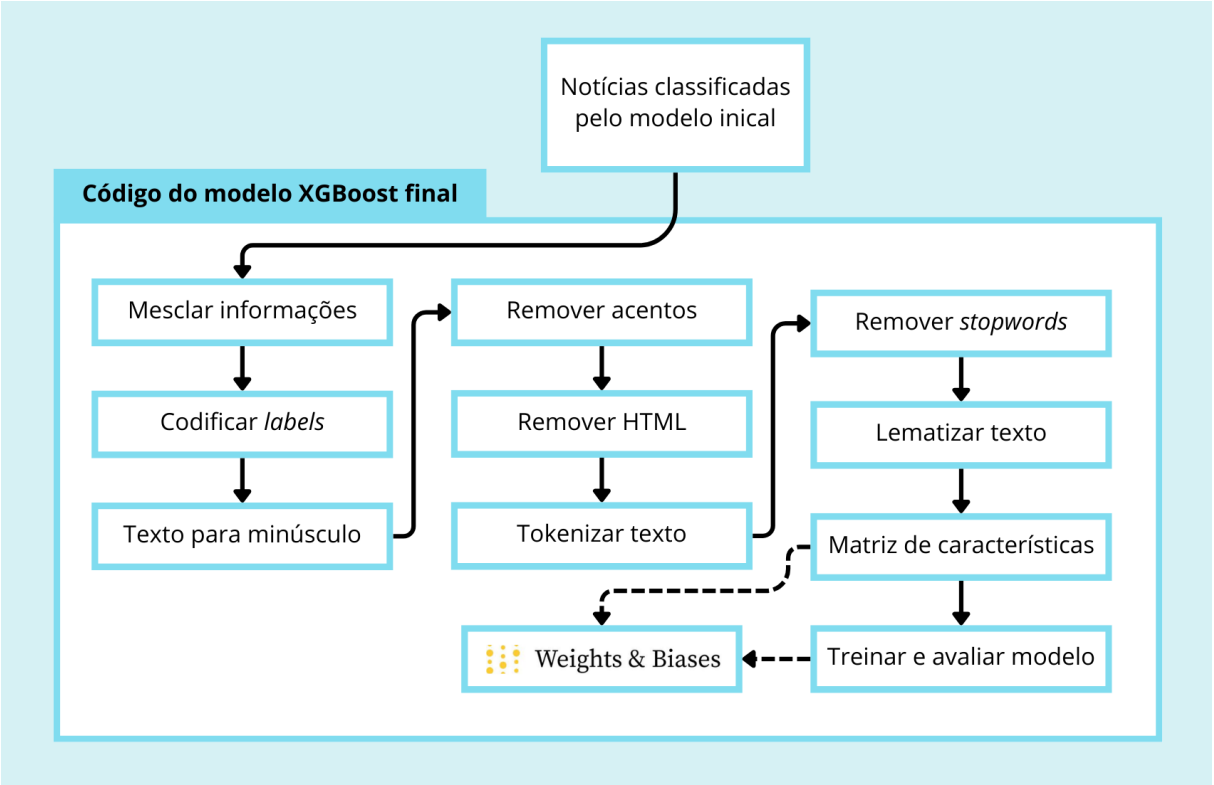
11. **Limpar e classificar demais notícias:** O preprocessamento do texto é igual ao descrito nos itens anteriores, com excessão da codifição das *labels*, já que as notícias que não foram classificadas manualmente não possuem rótulo. A etapa de obter a matriz de característica é feita com a instância obtida no passo (9), para manter a consistência dos dados a serem classificados pelo modelo. São selecionadas as notícias que possuem uma probabilidade maior que 80% de sua classificação estar correta, segundo o método `predict_broba()` da classe XGBoost. Esses novos dados são

então versionados e armazenados no W&B e no *Google Drive*, por meio do DVC. Eles servirão como entrada para o segundo modelo XGBoost.

3.1.3.2 Modelo Final

Neste modelo, que é o núcleo da abordagem com o algoritmo *XGBoost*, são realizados os mesmos procedimentos descritos até o item (8) da subseção 3.1.3.1. Os demais passos são feitos de forma diferente ou não existem. Esse fluxo é observado na Figura 8.

Figura 8 – Visão do código desenvolvido para o modelo XGBoost final.



Fonte: O autor, 2024.

As mudanças estão listadas abaixo.

- Conforme é possível observar, a matriz de características agora se comunica com o W&B. Essa matriz e a instância responsável por gerá-la são armazenadas nele. Esses elementos são utilizados na interação do usuário com o modelo, como é explicado na subseção 3.2.3.

- Em relação ao treinamento e avaliação do modelo, eles se comunicam com o W&B por dois motivos: salvar o modelo treinado e gerar gráficos interativos de forma automática a partir de certos parâmetros. As métricas que estão sendo observadas são as mesmas que no modelo inicial. São elas acurácia, precisão, *recall* e *F1 score*,

além da matriz de confusão. Ademais, não é preciso versionar ou salvar dados, pois não há alteração neles.

- A classificação de um novo conjunto de notícias, feita no passo (11) do primeiro modelo não é necessário, tendo-se em vista que este é o modelo final e todas as notícias foram rotuladas anterior.

3.2 Interação com os Modelos

A interação do usuário com os modelos desenvolvidos é intermediada pelo Gradio, que com a função `Interface()`, acrescida dos tipos de entrada e saída, e da função que irá ser chamadada quando o usuário interagir com o sistema, contrói um ambiente amigável e capaz de utilizar modelos de *machine learning*.

Dito isso, foram implementados três ambientes para lidar com os diferentes modelos. Eles estão hospedados no *Hugging Face*, site que permite o versionamento de código e integração com várias ferramentas, incluindo o Gradio.

Esta solução utiliza o W&B para carregar os modelos desenvolvidos e os artefatos voltados para o preprocessamento dos textos, como a instância que normaliza a representação numérica dos textos, nos modelos com *K-Means*, e a instância do PCA, que reduz a dimensionalidade dos dados, no modelo com *XGBoost*. Além dos itens citados, o W&B também versiona os conjuntos de dados dos modelos, garantindo uma alta eficiência e rapidez nesse processo.

A Figura 9 mostra as *interfaces* implementadas para cada modelo. A seguir, estão apresentadas a estrutura e funcionamento delas.

Os ambientes de interação com os modelos propostos, assim como os códigos utilizados para os seus respectivos desenvolvimentos, estão nos links abaixo:

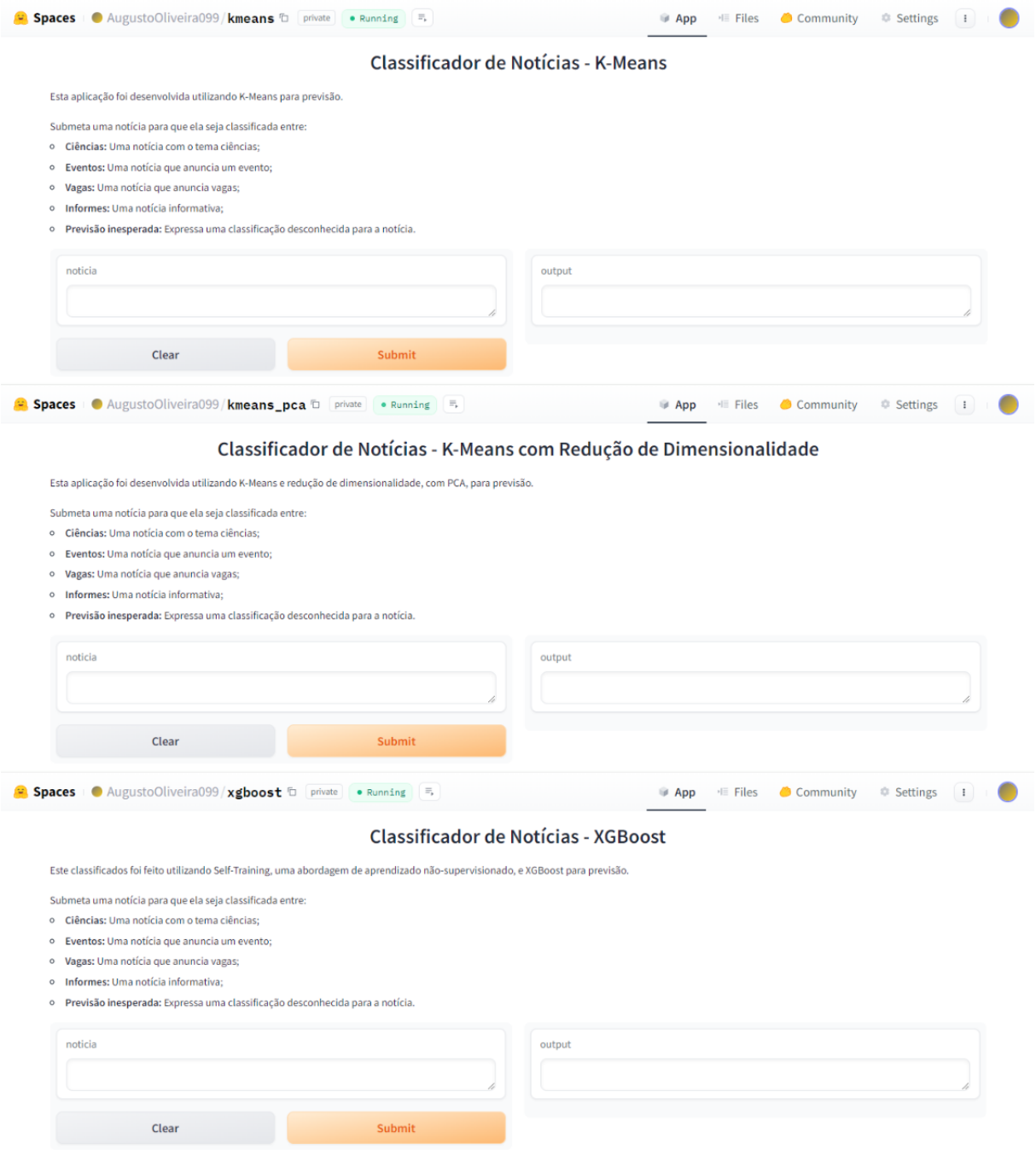
- *K-Means*.
- *K-Means* com redução de dimensionalidade.
- *XGBoost*.

3.2.1 K-Means

A Figura 10 ilustra o fluxo para estruturação e integração do ambiente que permite a interação com o modelo *K-Means* desenvolvido.

Quando o ambiente de interação, chamado pelo *Hugging Face* de "*space*", é reiniciado, acontecem os passos descritos na seção "Estruturação da *interface*", mostrada na Figura 10.

Figura 9 – Visualização do ambiente de intereção no *Hugging Face*.

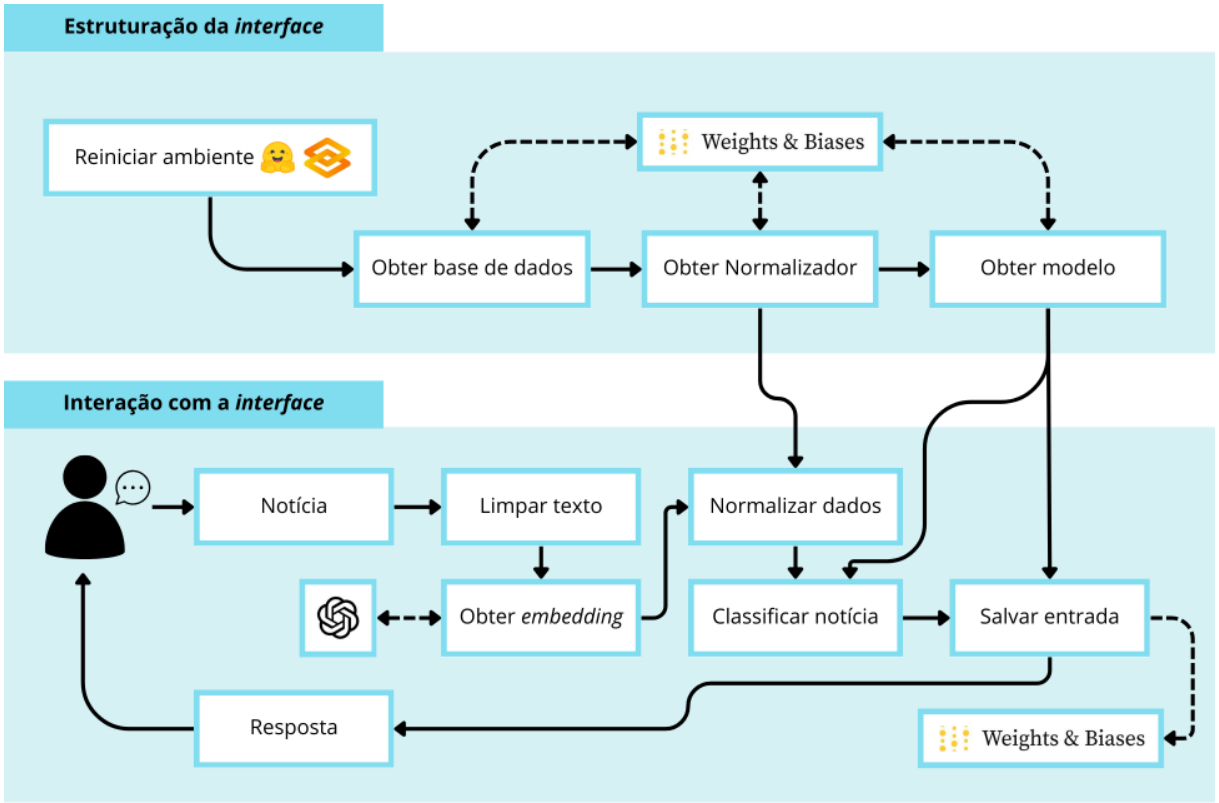


Fonte: O autor, 2024.

Isso ocorre quando há uma atualização no código ou quando o ambiente fica inativo e é preciso reiniciá-lo.

O fluxo começa pelo carregamento da base de dados salva no W&B, de forma manual, ao final do treinamento do modelo, no formato Comma-Separated Values (CSV), devido a problemas de desempenho do sistema com o DVC. Por fim, os artefatos salvos no W&B são carregados e ficam prontos para utilização. São eles a instância do normalizador de dados e o modelo.

Figura 10 – Fluxo para estruturação e interação com o modelo *K-Means*.



Fonte: O autor, 2024.

Na seção "Interação com a *interface*" é mostrado como o usuário pode se comunicar com o modelo treinado.

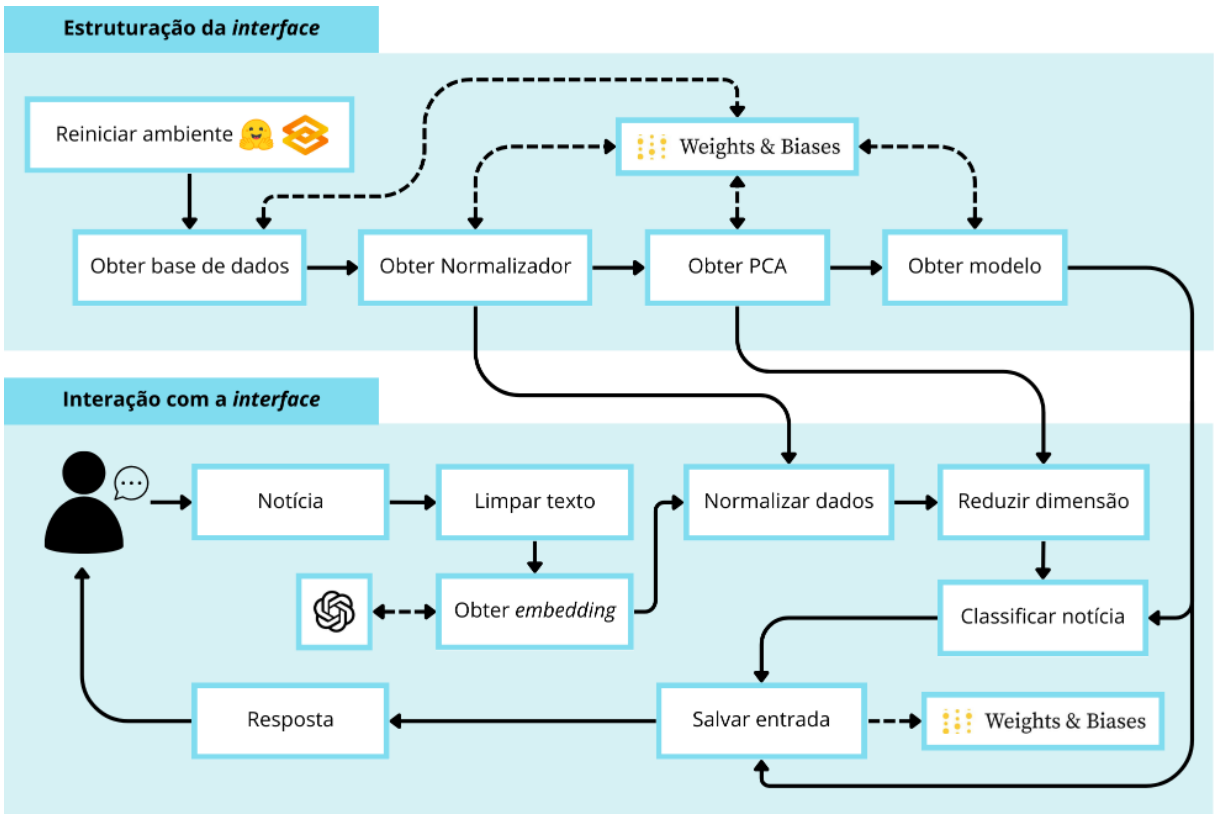
Quando o usuário solicita a classificação de uma notícia, é feita a mesma limpeza no texto decrita na subseção 3.1.1. Em seguida, o *embedding* do texto é solicitado à API da *OpenAI*. Então, é realizada a normalização dos dados, por meio do artefato obtido na seção anterior, e porterior classificação da notícia. Em seguida, há o salvamento da notícia e sua classificação na base de dados para posteriormente treinar um novo modelo.

Para a última etapa citada, é verificado, antes do salvamento, se existe alguma notícia na base de dados que seja similar a que foi classificada com base na comparação dos *embeddings*, caso não haja, a nova entrada é armazenada no W&B. Para essa verificação está sendo utilizada a métrica de distância textual "similaridade por cosseno", que verifica a similaridade de dois vetores. Neste caso, são comparados o vetor que representa o texto de entrada do usuário com cada um dos embeddings da base de dados. Isso é necessário pois usuários diferentes podem fazer a classificação do mesmo texto, ou modificar algumas palavras, porém essa informação não acrescenta valor ao modelo.

3.2.2 K-Means com redução de dimensionalidade

Para esta abordagem, os passos descritos na subseção 3.2.1 se aplicam, com a adição da redução de dimensionalidade. A instância ajustada aos dados é capturada por meio do W&B e utilizada entre as etapas de normalização e classificação da notícia com uso do modelo. Na seção "Interação com a *interface*", na Figura 11, é possível observar o que foi descrito.

Figura 11 – Fluxo para interação com o modelo *K-Means* que faz uso do PCA.

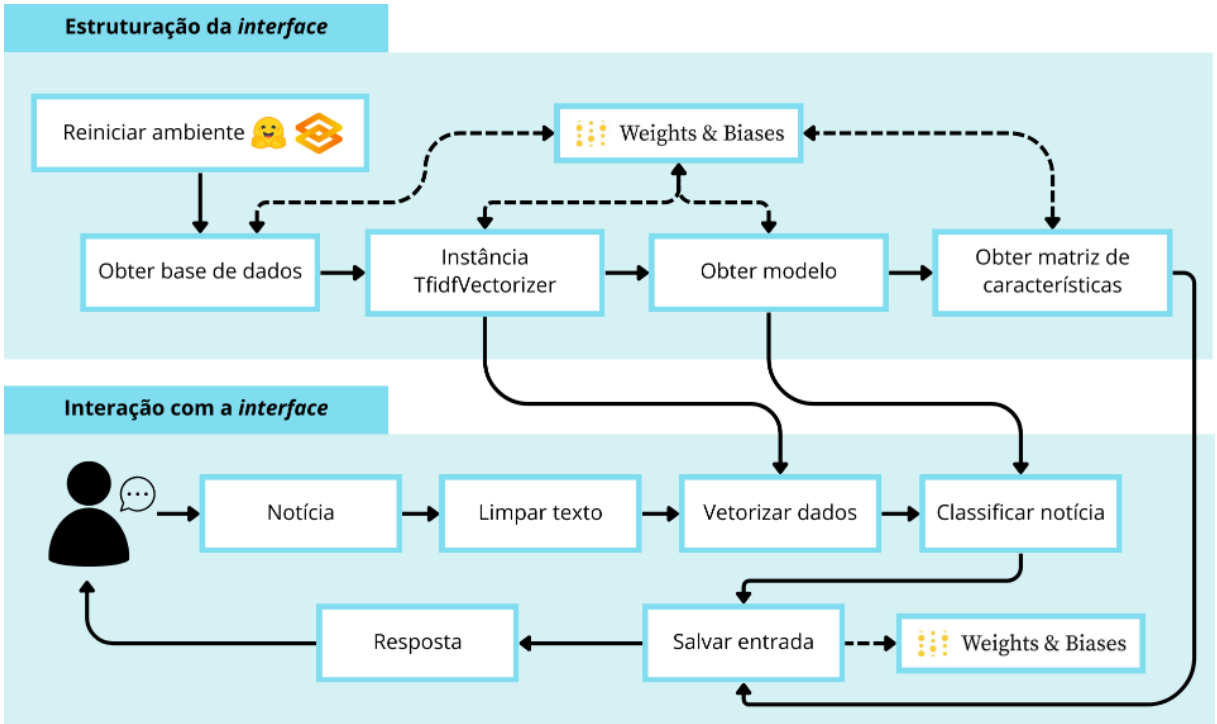


Fonte: O autor, 2024.

3.2.3 XGBoost

Para o modelo *XGBoost*, a implementação de sua interface é ligeiramente mais simples, já que ele não faz requisições para a API da *OpenAI*. Por essa razão, não é preciso armazenar o embedding da notícia, o que faz seu tempo de resposta ser consideravelmente menor que os outros modelos. O seu funcionamento está disposto na Figura 12.

Figura 12 – Fluxo para estruturação e interação com o modelo *XGBoost*.



Fonte: O autor, 2024.

A seção "Estruturação da *interface*" se inicia capturando a base de dados oriunda do treinamento. Em seguida são capturados os artefatos utilizados para classificação das notícias: a instância da classe *TfidfVectorizer* e o modelo treinado. Por fim, a matriz de características, gerada no treinamento do modelo a partir da instância citada.

Já na seção "Interação com a *interface*" existem algumas diferenças acentuadas em relação às outras abordagens. Quando o usuário solicita a classificação de uma determinada notícia, o processo de limpeza realizado é o mesmo descrito na subseção 3.1.3.1. Então, a instância do vetorizador é utilizada para obter a representação vetorial do texto. Em seguida, o modelo carregado do W&B classifica a notícia. Para salvar a entrada é feita uma comparação entre o vetor esparso da notícia e a matriz de características dos dados capturada na primeira etapa. Caso não haja similaridade, a nova entrada é salva no W&B, juntamente com a matriz de características atualizada. Por fim, a classificação da notícia é devolvida ao usuário, junto com a probabilidade da previsão estar correta.

4 RESULTADOS

Este capítulo tem como objetivo apresentar os resultados obtidos com o treinamento dos modelos *K-Means*, *K-Means* com redução de dimensionalidade via PCA e *XGBoost*, de forma individual e comparativa. Além disso, serão avaliadas as *interfaces* de interação com os modelo mencionados.

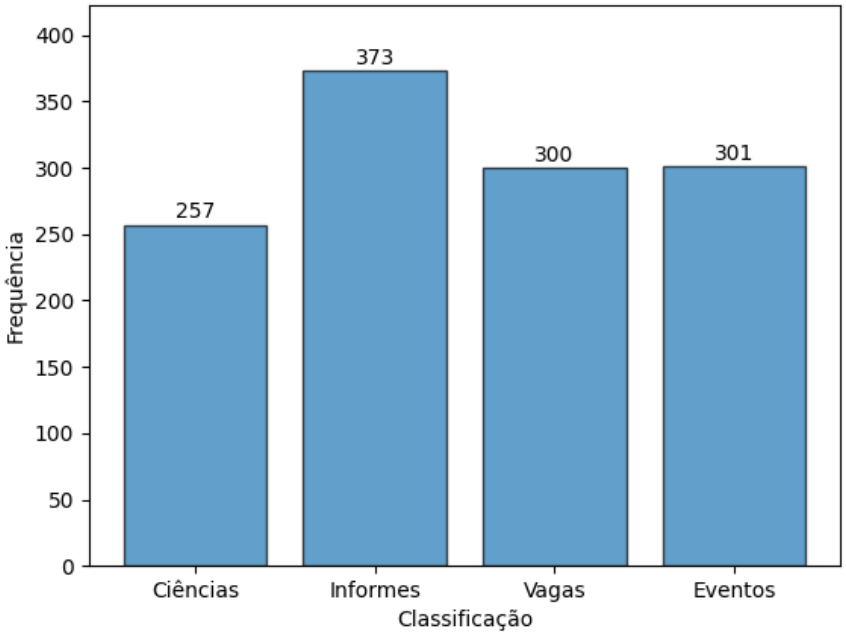
4.1 Modelos propostos

Os resultados dos modelos implementados neste trabalho são expostos nas subseções seguintes, que detalham os hiperparâmetros de cada um deles e descrevem as métricas obtidas ao final do processo. Por fim, é feita a comparação entre os resultados individuais dos modelos.

É importante ressaltar que em contato com a Agência de Comunicações da Universidade Federal do Rio Grande do Norte (AGECOM/UFRN), responsável por escrever e publicar as notícias tratadas neste trabalho, foi informado que existem quatro classes para os textos: eventos, informes, vagas e ciências. Sendo assim, o número de *clusters* para os modelos que utilizam o algoritmo K-Means é estritamente quatro.

Nesse sentido, os modelos utilizaram 21777 notícias obtidas por meio do endpoint disponibilizado pela STI. Dentre elas, 1231 foram separadas e classificadas manualmente com o objetivo de treinar o modelo inicial do XGBoost e analisar o resultado de todos os modelos. Mantendo as mesmas notícias para todas as análises de eficiência, é possível fazer uma comparação precisa. Essas notícias estão distribuídas conforme mostra a Figura 13.

Figura 13 – Distribuição por tema das notícias classificadas manualmente.



Fonte: O autor, 2024.

A quantidade desbalanceada de notícias por tema no conjunto acima se dá pela escassez de textos classficados como pertencentes ao tema "Ciências", dentre mais de 4000 notícias analisadas, apenas 257 se enquadravam nele. Para não ficar tão desbalanceado, procurei manter essa quantidade em torno de 300 notícias por tema.

Ademais, limitações computacionais fizeram com que ao utilizar algoritmos de *hyperparameter tuning* o computador travasse e fosse preciso reiniciá-lo, sendo assim, todos os modelos passaram por vários testes até serem encontrados hiperparâmetros que produzissem o melhor resultado.

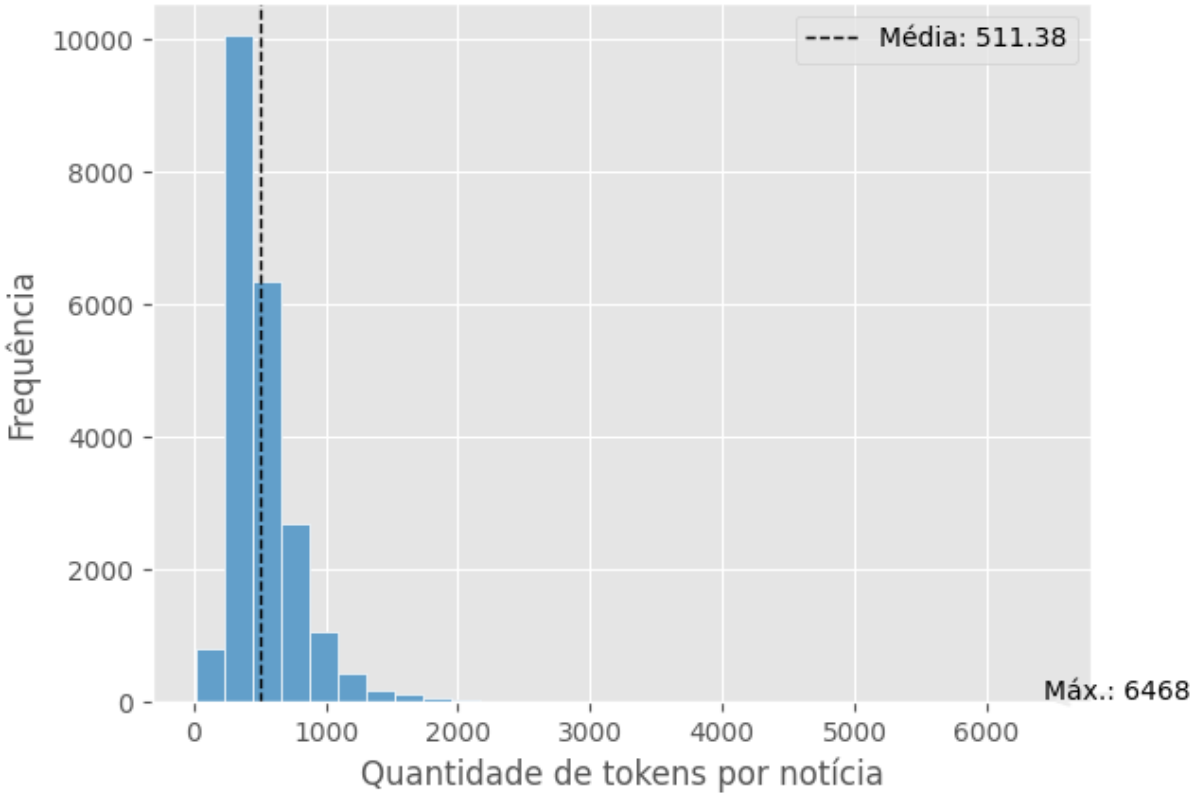
4.1.1 K-Means

Para o modelo *K-Means*, a nuvem de palavras do conjunto de textos utilizados em seu treinamento está disposta na Figura 14.

Figura 14 – Nuvem de palavras dos dados utilizados no modelo *K-Means*.

da UFRN
para o
Rio Grande
Universidade Federal
Norte UFRN
por meio
dia de
da Universidade
Conteúdo O
de pesquisa
Departamento de
o evento
e de
Programa de
de Ciência
curso de
Pós Graduação
de saúde
podem ser
Centro de
o projeto
de ensino
e mail
além de
de Gestão
sobre o
Graduação em
Ensino Superior
de Pó
e da
de setembro
de extensão
o objetivo
o dia
até o
de agosto
cursos de
de maio
em Saúde
na UFRN
de graduação
Pró Reitoria
que o
área de
de trabalho
Os interessados
todos o
Gestão de
Título UFRN
De acordo
processo seletivo
de Pessoa
o desenvolvimento
Mais informações
inscrições para
Outras informações
pesquisa e
Atividades Acadêmicas
de Atividade
de Educação
Norte
entre o
pode ser
o tema
ufrn.br
quarta feira
o professor
de outubro
de novembro
de março
de julho
ser feita
de um
Nacional de
Escola de
de forma
segunda feira
sexta feira
terça feira
dias e
tem como
o curso
processo de
partir da
o processo
em parceria
quinta feira
O Programa
Ciências da
da Educação
Laboratório de
Brasil
o grupo
pelo o
toda
mais de
Ministério da
Núcleo de
Macroeconomia Digital
Ciências Sociais
que se
áreas de
de Tecnologia
e Tecnologia
O resultado
Reitoria de
UFRN realiza
bem como
UFRN e
de Natal
na área
período de
os dia
os alunos
meio da
um dos
desenvolvimento de
informações podem
informações sobre
como objetivo
Grupo de
Grupo de Pesquisa
Integrado de
graduação e
Além disso
o edital
de Música
Inovação
de janeiro
de abril
de fevereiro
da instituição
Ministério da
Saúde
abertas até
de dezembro
Atividade
Arte
durante
Instituto
Nacional de
de vida
de junho
como o
também
Ensino Superior
de uma
Estudo
Sistema Integrado
Ciência
Comunicação
Escola
inscrições abertas
meio de
estudantes de
se inscrever
Ministério da
Saúde
Nacional de
Ciência
Tecnologia
Inovação
Engenharia
Arte
Música
Natal
Pessoa
Candidatos
Desenvolvimento
Informações
Outras informações
Atividades Acadêmicas
Pesquisa e
Graduação em
Estudo
Sistema Integrado
Ciência
Comunicação
Escola
Educação
Norte
Saúde

Figura 15 – Frequência da quantidade de *tokens* por notícia.



Fonte: O autor, 2024.

Conforme explicado no passo 3 da subseção 3.1.1, a quantidade máxima de tokens que o modelo da *OpenAI* aceita é 8191. Conforme visto na Figura 15, o número máximo de tokens em uma dada notícia é 6468, logo não foi preciso remover nenhum texto.

Ademais, foram utilizados no treinamento do modelo os hiperparâmetros apresentados na Tabela 1.

Tabela 1 – Hiperpâmetros para o modelo K-means.

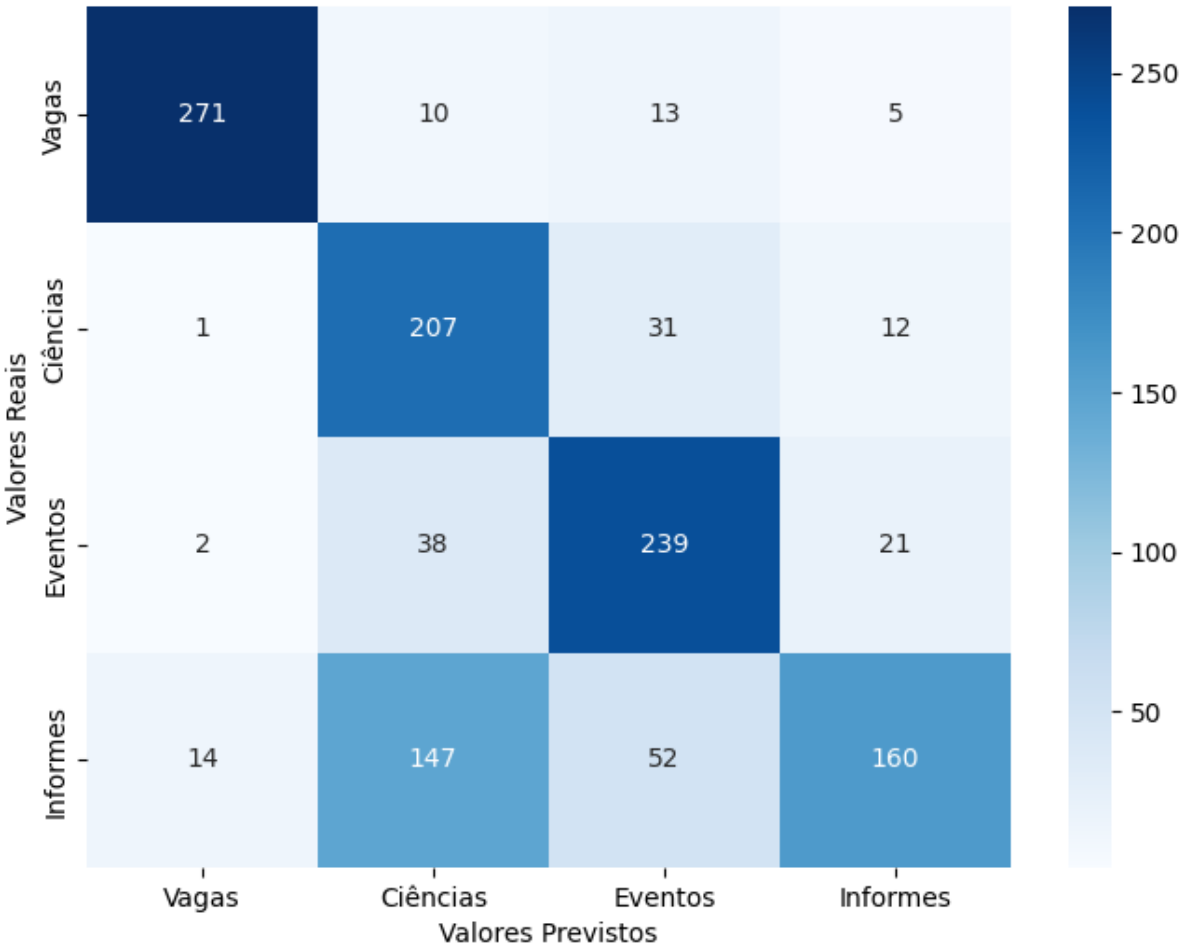
Hiperparâmetro	**Valor**
init	random
n_init	30
max_iter	500
tol	0.001
random_state	3

Fonte: O autor, 2024.

Para treinar o modelo com os hiperparâmetros apresentados, são utilizadas todas as notícias, excluindo aquelas que foram classificadas manualmente, já que elas servem como o conjunto de dados de teste para o modelo.

Após testar o modelo com os textos citados, foi obtida a matriz de confusão apresentada na Figura 16.

Figura 16 – Matriz de confusão do modelo *K-Means*.



Fonte: O autor, 2024.

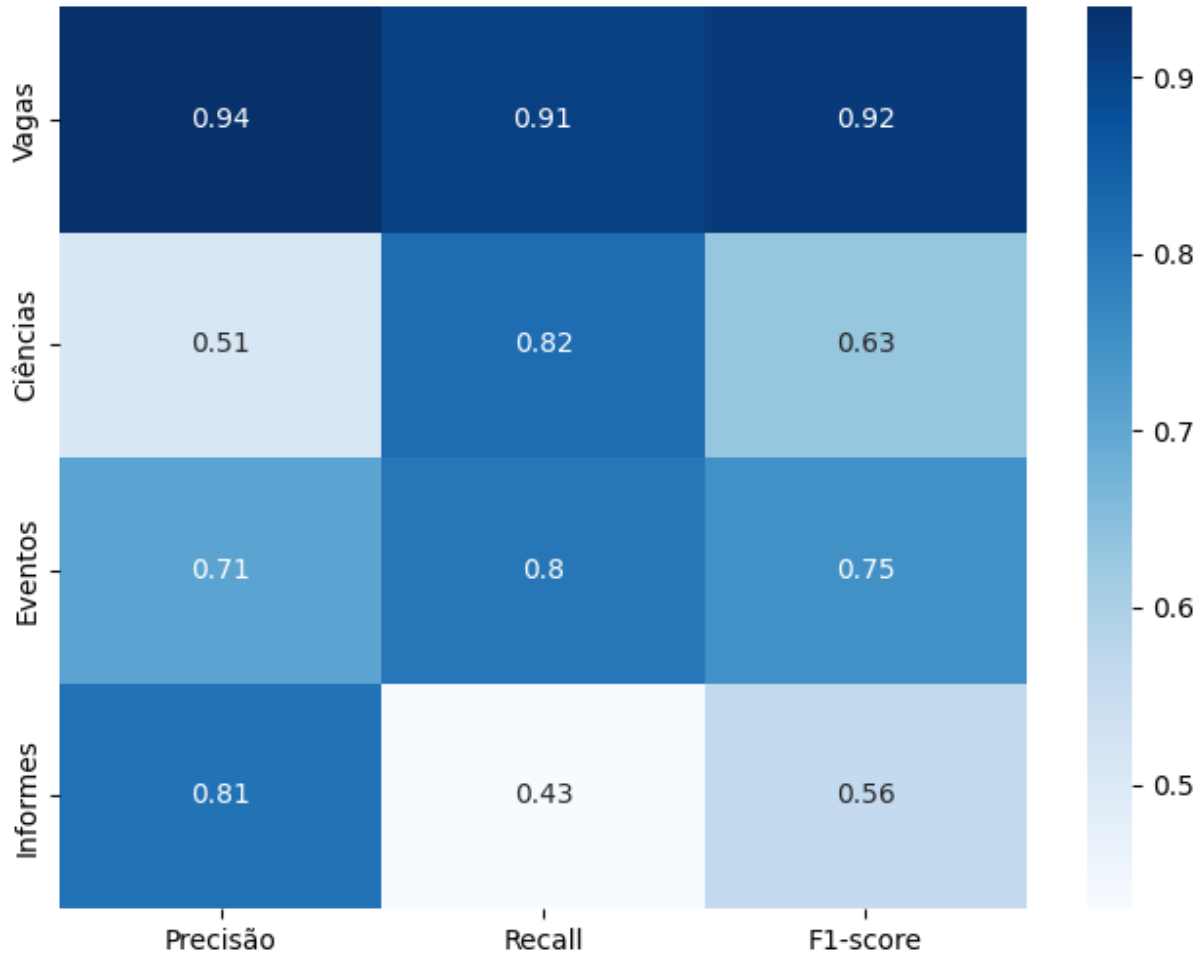
A partir da matriz de confusão, é possível calcular várias métricas importantes para medir o quão acertivo o modelo é. Neste trabalho são avaliadas a acurácia, a precisão, a sensibilidade, ou *recall*, e o *F1-score*.

A acurácia mede a proporção de previsões corretas em relação ao total de previsões. A precisão refere-se à proporção de verdadeiros positivos em relação às previsões positivas, ou seja, é a relação entre a quantidade de previsões corretas de uma amostra pertencer à classe observada em relação a quantidade total de intâncias classificadas como sendo dessa classe. O *recall* mede a proporção de verdadeiros positivos em relação ao total de amostras positivas reais, que por sua vez, dizem respeito aos exemplos que são realmente de uma determinada classe observada. O *F1-Score* é a média harmônica entre a precisão e a sensibilidade, oferecendo um equilíbrio entre elas.

A acurácia obtida após o treinamento foi de aproximadamente 71,71%. Na Figura 17

é mostrado o relatório de classificação do modelo em questão.

Figura 17 – Relatório de classificação para o modelo *K-Means*.



Fonte: O autor, 2024.

Uma alta precisão indica que poucas notícias de classes diferentes da que se está sendo observada (classe positiva) foram classificadas como pertencentes a ela (falso positivo). Por outro lado, o *recall*, ou sensibilidade, mede o quão bem notícias da classe positiva estão sendo classificadas como sendo de fato desta classe. As classificações incorretas nesse contexto são chamadas de falso negativo. Já o *F1-score* é a média harmônica entre a precisão e o *recall*, o que proporciona um equilíbrio entre essas duas métricas e evita que o foco excessivo em uma delas comprometa a outra.

Conforme a acurácia evidencia, o modelo possui um bom resultado no geral. A classe "vagas" obteve um resultado superior às outras classes em todas as métricas citadas, tendo-se em vista que no texto existem palavras e expressões que evidenciam o tema da notícia, como "vaga" e "processo seletivo". Para a classe "eventos", os textos mencionam datas e períodos no qual os eventos vão ocorrer, o que possibilita uma boa acertividade. Entretanto, para os outros temas aqui retratados, essa divisão não é tão clara, o que leva,

consequentemente, a classificações incorretas.

É importante analisar de forma isolada alguns pontos, como a baixa precisão da classe "ciências" e o baixo *recall* da classe "informes". A razão disso fica evidente ao se analisar a matriz de confusão na Figura 16. Notícias que são do tema "informes" por vezes são erroniamente classificadas como textos que discorrem sobre ciências, consequentemente, fazem a sensibilidade e a precisão de "informes" e "ciências" serem baixas, respectivamente. Isso ocorre por esses textos não possuírem palavras que façam sua clara distinção.

O *F1-score* mostra um desempenho fraco do modelo ao classificar notícias do tema "informes", prejudicado pelo baixo *recall*. Ele denota, também, um desempenho mediado para "ciências", que é prejudicado pela baixa precisão. Enquanto os temas "eventos" e "vagas" possuem uma boa e ótima média harmônica, respectivamente.

4.1.2 *K-Means* com redução de dimensionalidade

Para o modelo *K-Means* com o acréscimo da redução de dimensionalidade, a nuvem de palavras e a distribuição dos tokens são iguais, dado que as notícias utilizadas para o treinamento e teste são as mesmas. Não foi gasto nenhum valor para obter os *embeddings* dos textos, uma vez que eles foram aproveitados do modelo explicado na subseção 4.1.1.

No que diz respeito ao PCA, os hiperparâmetros utilizados na criação da instância responsável pela redução de dimensionalidade estão dispostos na Tabela 2.

Tabela 2 – Hiperpâmetros para a instância do PCA.

Hiperparâmetro	**Valor**
n_components	2
svd_solver	randomized
random_state	42

Fonte: O autor, 2024.

Para o hiperparâmetro *n_components*, o valor precisa ser 2, umas vez que o objetivo é plotar o resultado dos *clusters* em um gráfico de duas dimensões. Quanto ao modelo *K-Means* propriamente dito, os hiperparâmetros utilizados estão na Tabela 3.

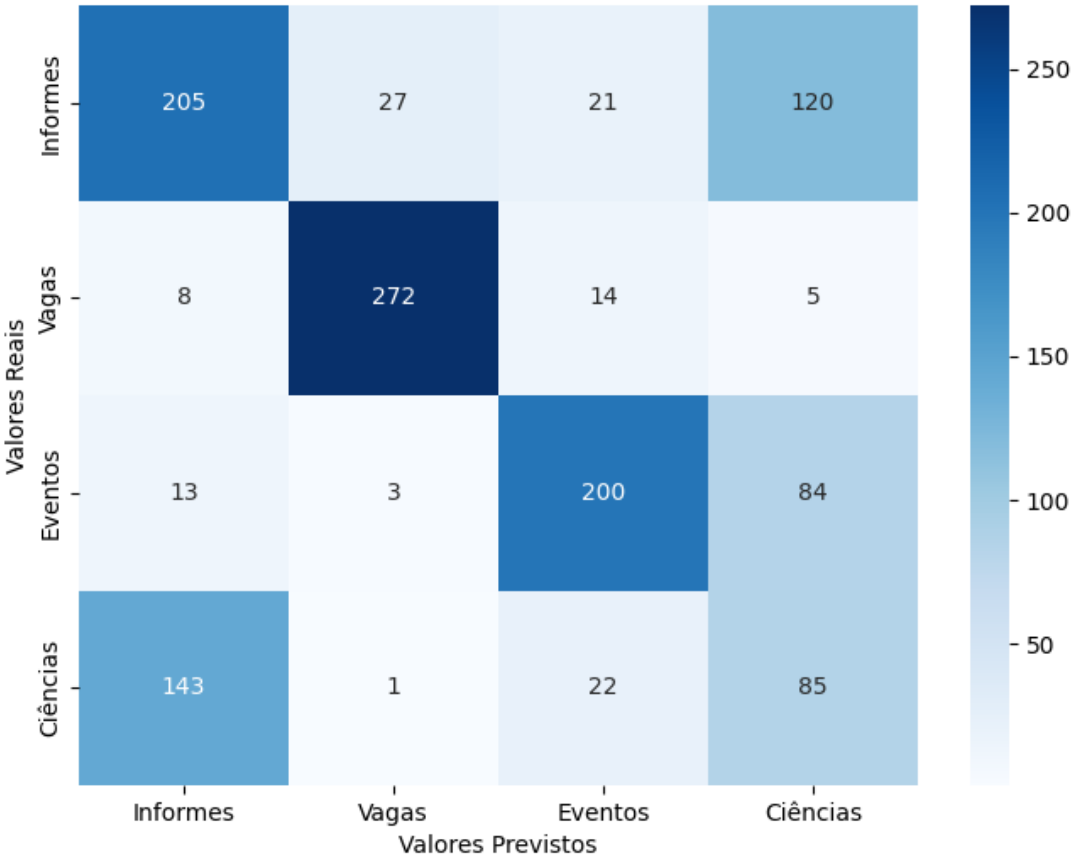
Tabela 3 – Hiperpâmetros para o modelo K-means com redução de dimensionalidade.

Hiperparâmetro	**Valor**
init	random
n_init	30
max_iter	500
tol	0.1
random_state	42

Fonte: O autor, 2024.

Os hiperparâmetros utilizados para este modelo e para o *K-Means* foram os mesmos, mudando apenas os valores para "*tol*" e "*random_state*". Em posse do modelo treinado, foi possível testá-lo com as notícias classificadas manualmente. A matriz de confusão resultante desse processo está na Figura 18.

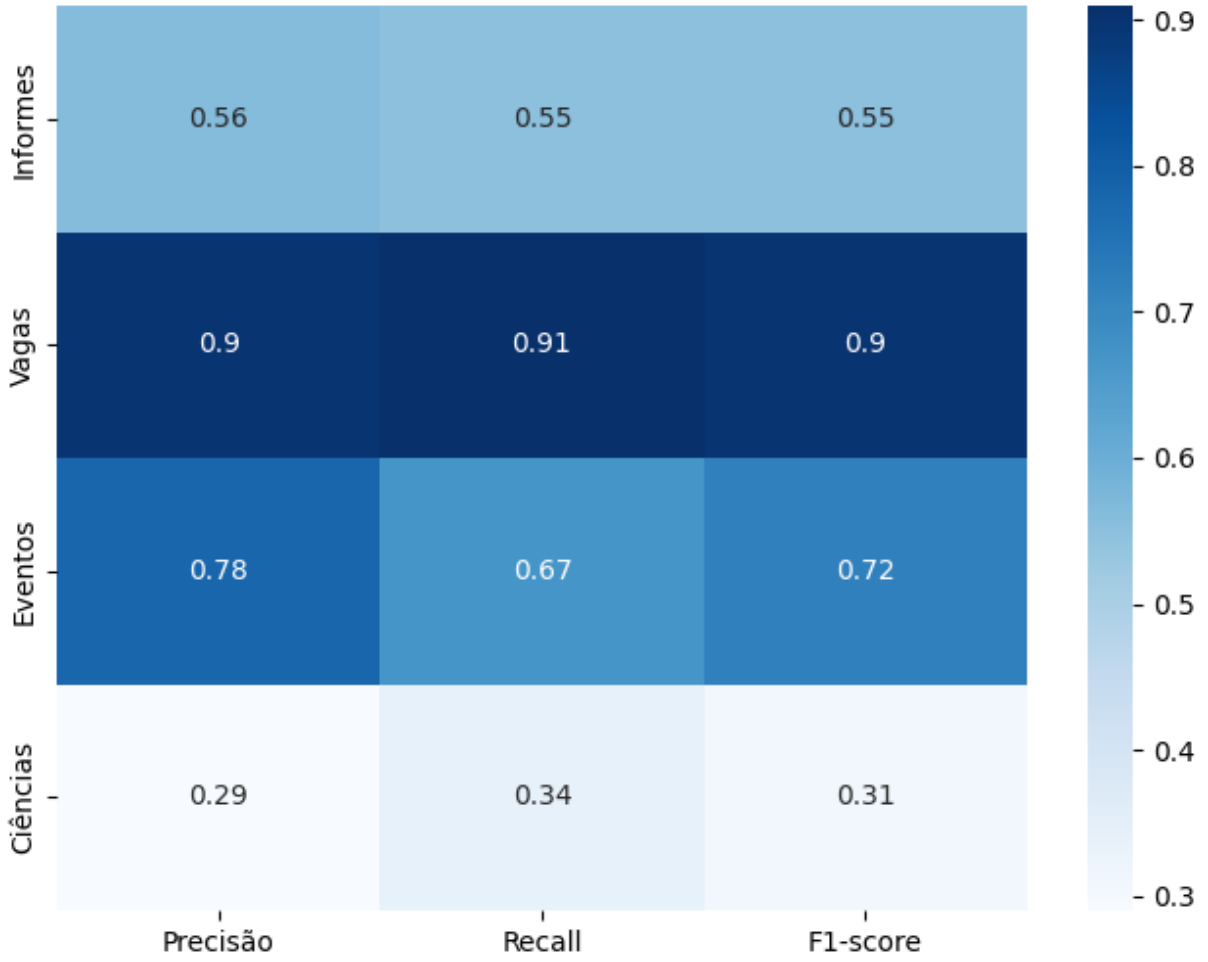
Figura 18 – Matriz de confusão do modelo *K-Means* com redução de dimensionaliade.



Fonte: O autor, 2024.

O modelo em questão obteve uma acurácia de 62,31%. Além disso, o restante das métricas estão dispostas na Figura 19.

Figura 19 – Relatório de classificação para o modelo *K-Means* com PCA.

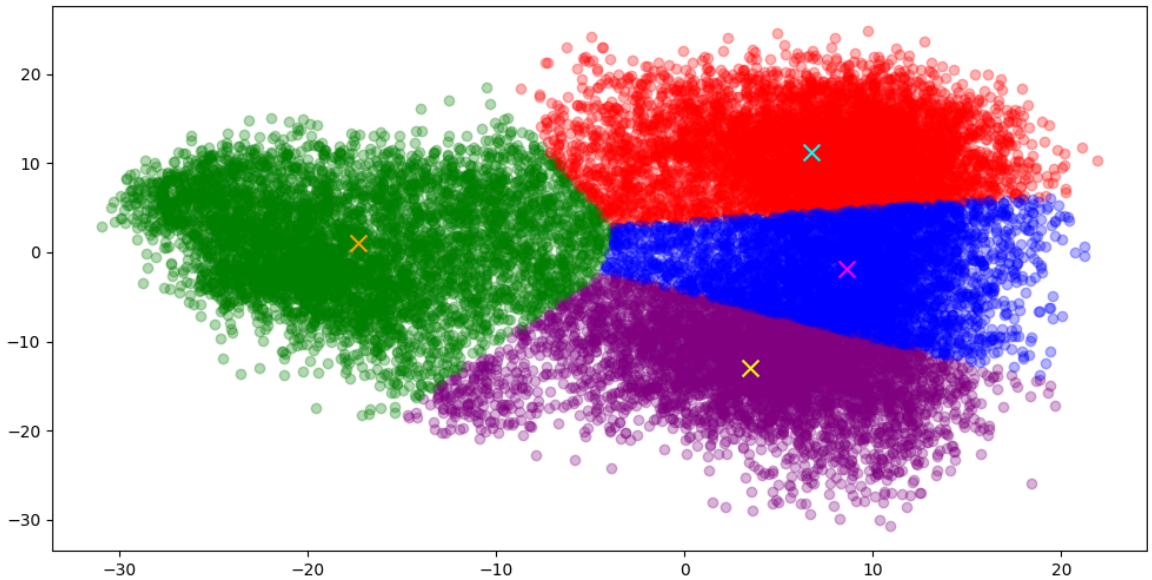


Fonte: O autor, 2024.

É notável o fraco desempenho do modelo para a classe "ciências". Ele apresenta baixa precisão e baixo *recall*, o que leva a um valor reduzido de *F1-score*. Como dito na subseção 4.1.1, as notícias com temas "ciências" e "informes" se confundem. Para este modelo, mais da metade das notícias de teste que falam sobre ciências são classificadas como sendo da classe "informes", resultando em um *recall* de 34%. Além disso, textos de outros temas muitas vezes são classificados como "ciências" - a não ser pelas notícias a respeito de vagas -, o que gera uma precisão de 29%. Os informes possuem um problema similar relacionado, também, ao tema "ciências", que leva a resultados medianos.

Em posse do modelo treinado e dos componentes bi-dimensionais das notícias, é possível visualizar os *clusters* formados, conforme mostra a Figura 20.

Figura 20 – Visualização dos *clusters* identificados usando PCA.



Fonte: O autor, 2024.

Quando um modelo utiliza o PCA para redução de dimensionalidade, é possível medir a quantidade de informação perdida nesse processo. Cada componente principal captura uma fração da variabilidade presente nos dados originais. O PCA seleciona os componentes que retêm a maior parte da variância dos dados, descartando os menos relevantes e, com eles, parte da informação.

O PCA fornece a variância explicada por cada componente principal. Ao somar a variância capturada por todos os componentes selecionados, obtém-se a variância explicada. O complemento desse valor corresponde à variância perdida. No caso analisado, os valores para essas métricas foram 7,87% e 92,13%, respectivamente.

Dessa forma, observa-se uma grande perda de informação ao reduzir os embeddings para duas dimensões. Esse fato impacta negativamente nas métricas do modelo, como evidenciado nesta seção.

4.1.3 XGBoost

Nesta subseção serão apresentados os resultados para os dois modelos que utilizam XGBoost em conjunto com a abordagem de aprendizado semi-supervisionado *Self-Training*.

4.1.3.1 Modelo inicial

O modelo inicial é responsável por classificar apropriadamente o restante das notícias que não possuem rótulo. A nuvem de palavras para o conjunto de dados classificados manualmente está na Figura 21.

Figura 21 – Nuvem de palavras dos dados utilizados no modelo XGBoost inicial.

Fonte: O autor, 2024.

Este modelo foi treinado com 90% dos dados para treinamento e os outros 10% para teste. Os hiperparâmetros utilizados para isso são os apresentados na Tabela 4.

Tabela 4 – Hiperpâmetros para o modelo XGBoost inicial.

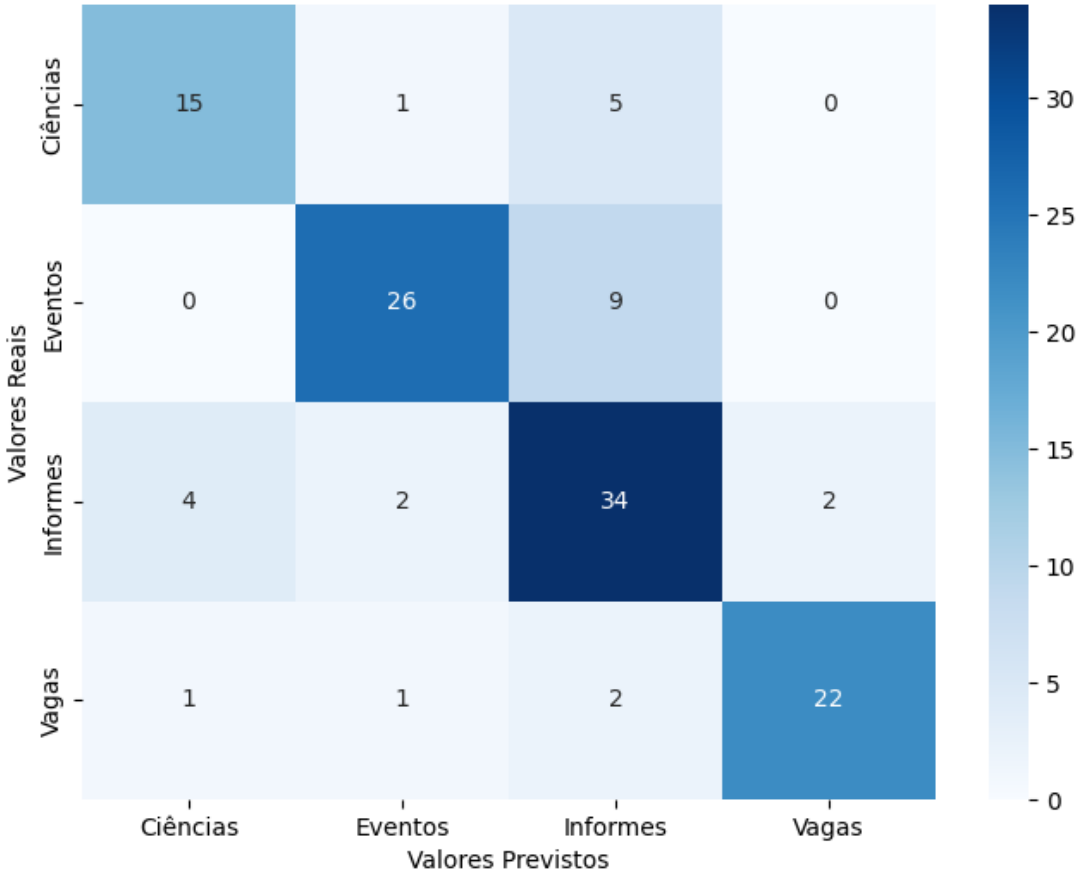
Hiperparâmetro	**Valor**
eval_metric	mlogloss
random_state	42

Fonte: O autor, 2024.

Foi encontrada uma acurácia no conjunto de treinamento de 100%, enquanto no conjunto de teste foi igual a 78,23%. Isso mostra um sobreajuste do modelo. Todavia, quando o modelo foi ajustado com hiperparâmetros que mitigam esse comportamento de ajuste perfeito, com a acurácia do conjunto de treinamento aproximadamente igual a 89% e a de teste 71%, o resultado do modelo final se mostrou pior. Mesmo alterando a proporção dos dados de treinamento e teste, a alta variância ainda persistiu. Sendo assim, foi optado por manter o modelo com sobreajuste.

A matriz de confusão do modelo para os dados de teste é mostrada na Figura 22.

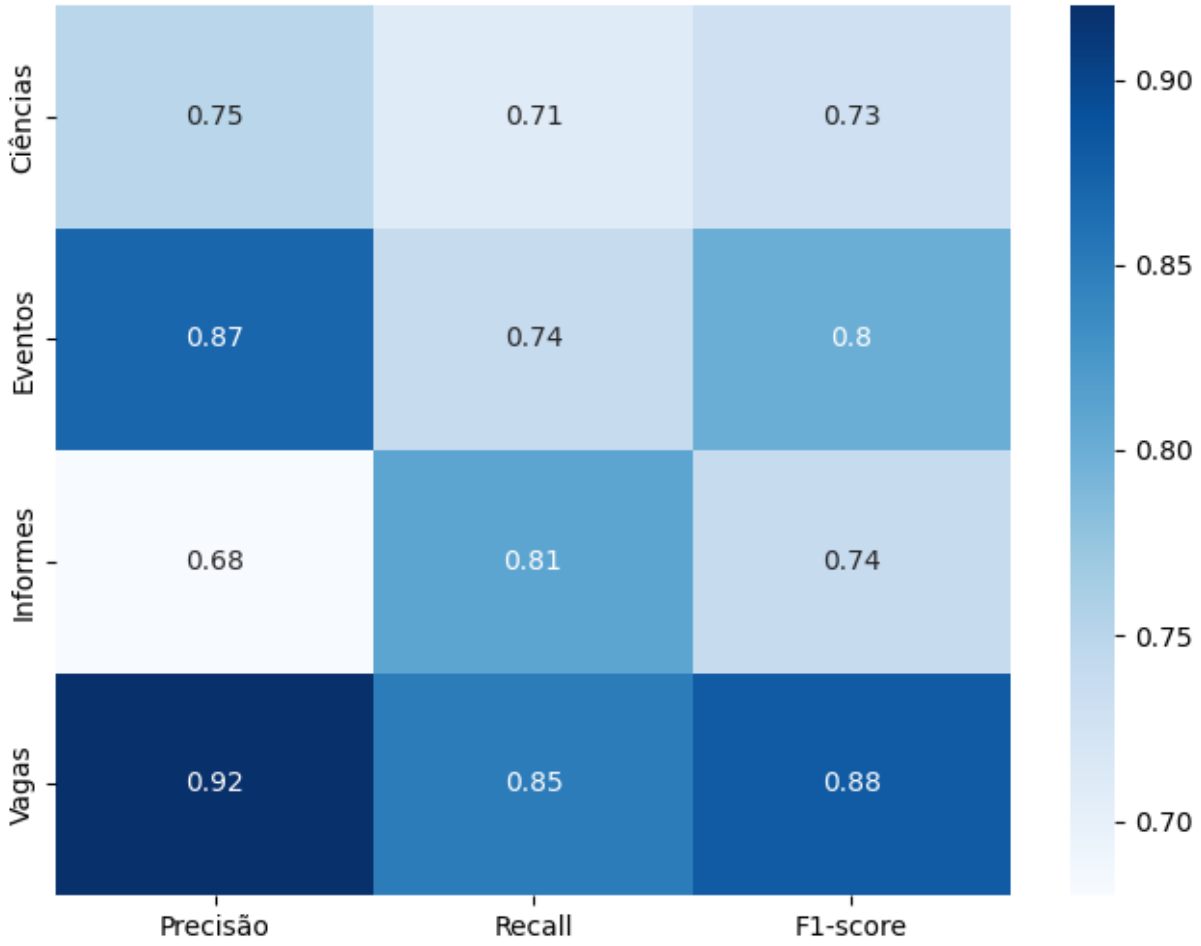
Figura 22 – Matriz de confusão do modelo XGBoost inicial.



Fonte: O autor, 2024.

Além disso, o relatório de classificação está sendo apresentado na Figura 23.

Figura 23 – Relatório de classificação para o modelo XGBoost inicial.



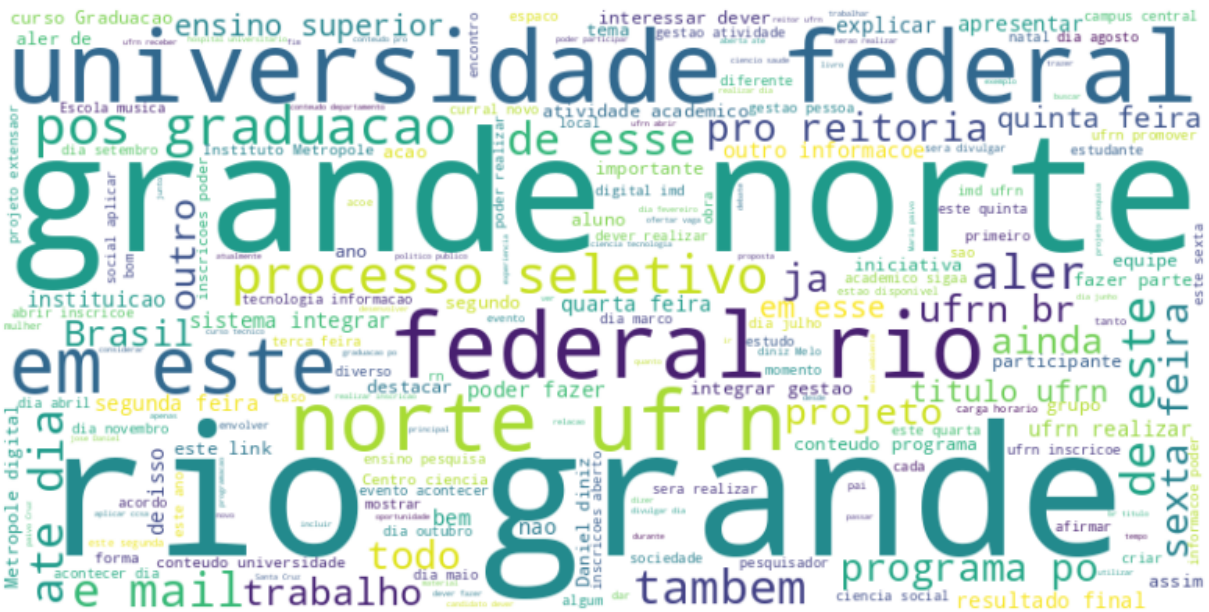
Fonte: O autor, 2024.

Como dito nas análises anteriores, as notícias classificadas como "ciências" se confundem com aquelas que são classificadas como "informes". Além disso, algumas notícias sobre eventos não possuem um padrão bem definido, também se confundindo com informes. Isso explica os resultados mostrados no relatório de classificação.

4.1.3.2 Modelo final

Este é o modelo de interesse, treinado com muitas notícias. É com ele que o usuário interage em produção. Os textos classificados pelo modelo inicial que possuem uma probabilidade maior ou igual a 80% da rotulação estar correta são utilizados para o treinamento do modelo em questão.

Figura 24 – Nuvem de palavras dos dados utilizados no modelo XGBoost final.



Fonte: O autor, 2024.

Para o conjunto de dados disponível, a quantidade de notícias que se enquadram na condição citada é de 14871. A nuvem de palavras referente a elas é apresentada na Figura 24. Como é possível observar, algumas palavras se destacam, em espcial o nome da universidade.

Já os hiperparâmetros utilizados na instância do XGBoost estão na Tabela 5.

Tabela 5 – Hiperpâmetros para o modelo XGBoost final.

Hiperparâmetro	**Valor**
learning_rate	0.2
max_depth	6
reg_alpha	5
reg_lambda	5
subsample	0.5
colsample_bytree	0.7
eval_metric	mlogloss
random_state	42
objective	multi:softmax
num_class	4

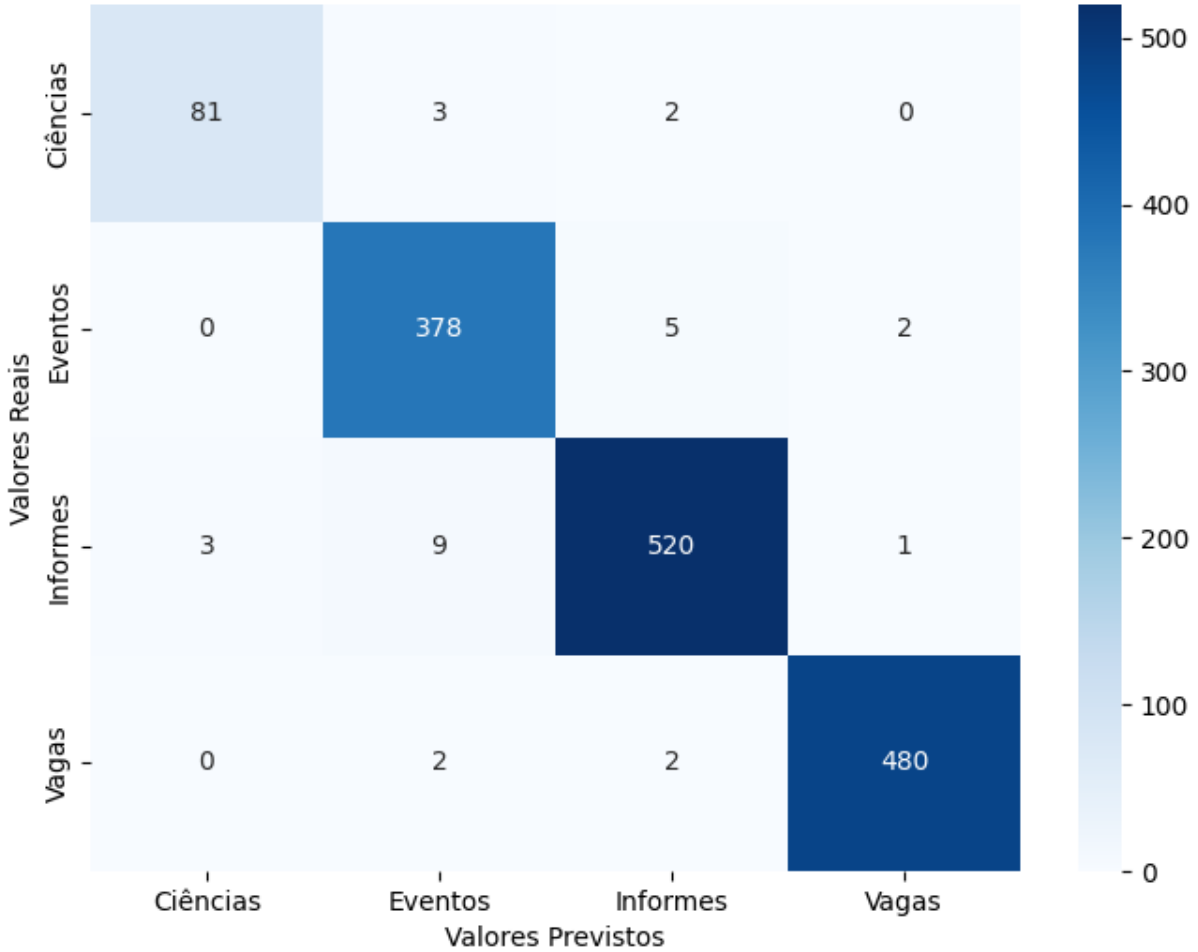
Fonte: O autor, 2024.

Das 14871 notícias citadas, 90% foi para treinamento e 10% para teste. Como as notícias classificadas manualmente foram utilizadas no treinamento do modelo inicial, o modelo final possui um certo enviesamento e seu resultado pode não ser totalmente confiável.

Sendo assim, serão abordados ambos os resultados. Para o conjunto de treinamento, o modelo apresentou acuária de 99,84%.

No conjunto de teste, a acurácia foi de 98,05%, cuja matriz de confusão e relatório de classificação estão exibidos na Figura 25 e na Figura 26, respectivamente.

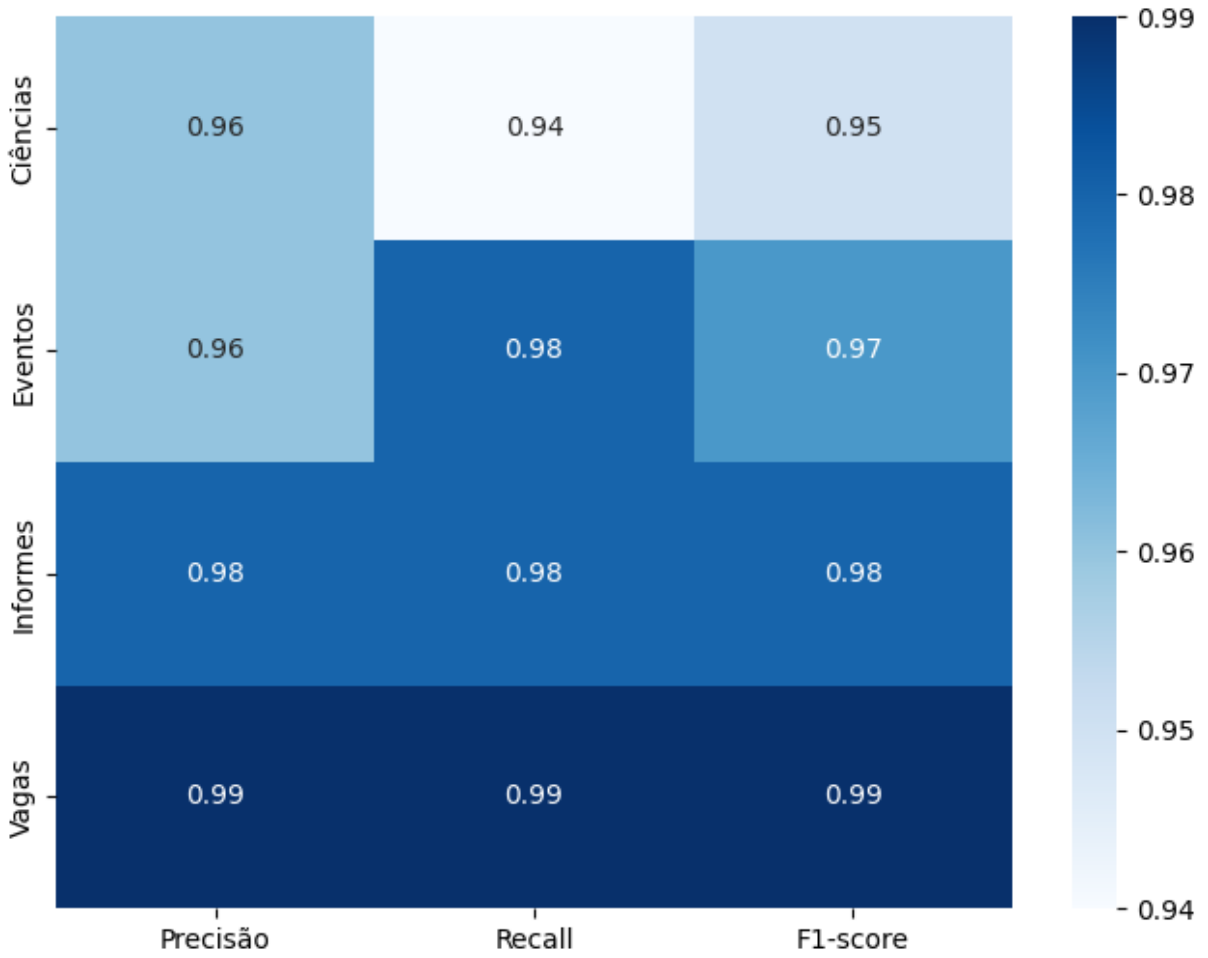
Figura 25 – Matriz de confusão do modelo XGBoost final para dados de teste.



Fonte: O autor, 2024.

Os temas das notícias estão desequilibrados, mas demonstra bem como os dados de fato são, uma vez que a proporção de textos que discorrem sobre ciências é baixa, enquanto a proporção para os outros temas é maior, com destaque para “informes”.

Figura 26 – Relatório de classificação para o modelo XGBoost final para dados de teste.

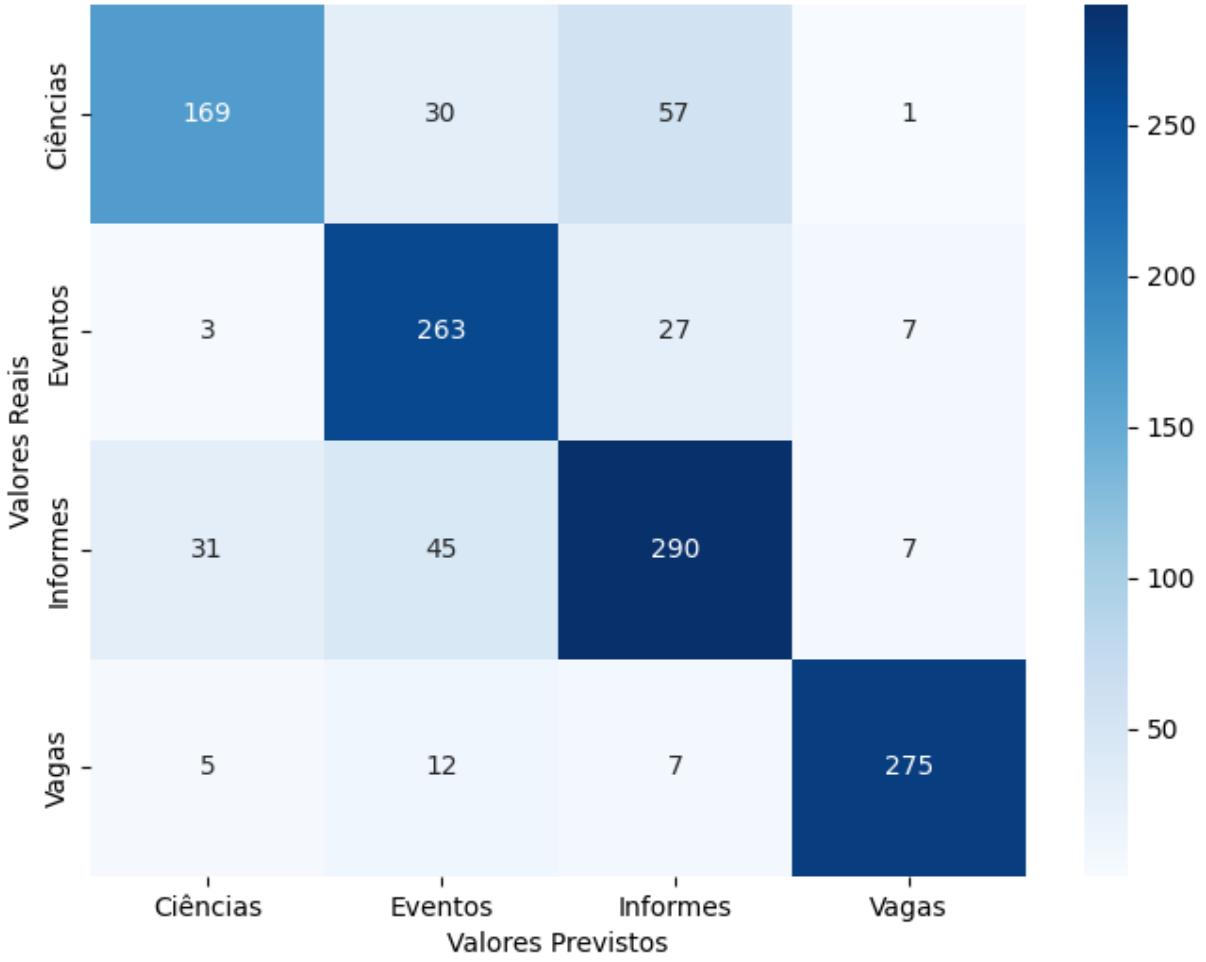


Fonte: O autor, 2024.

É notório o alto desempenho do modelo para todas as classes, como mostra o relatório de classificação, com destaque para "vagas".

No que diz respeito à utilização dos textos classificados manualmente para testar o modelo, o teste apresentou acurácia de 81,12%. A matriz de confusão é apresentada na Figura 27.

Figura 27 – Matriz de confusão do modelo XGBoost final para notícias classificadas manualmente.



Fonte: O autor, 2024.

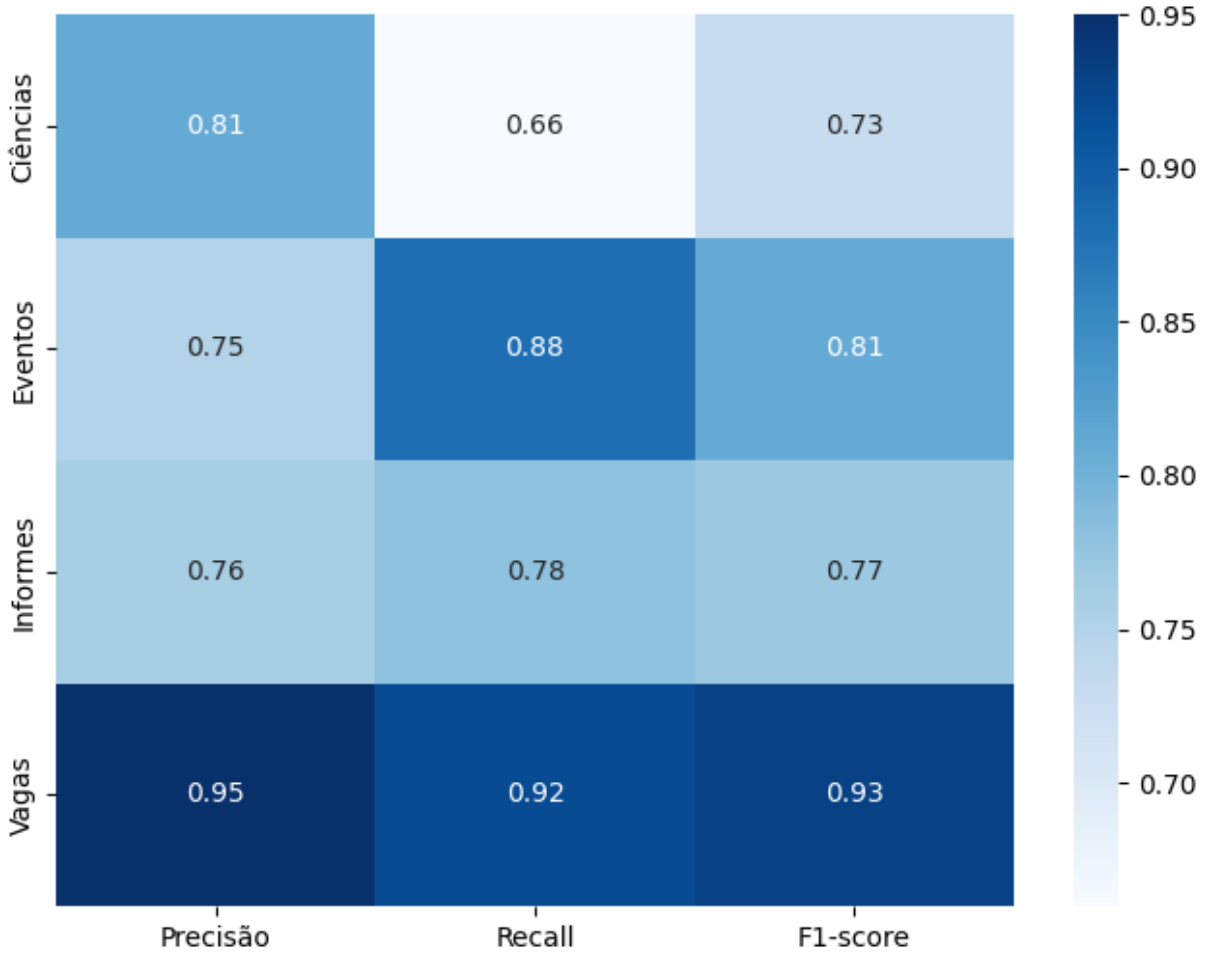
Para este modelo, não houveram grandes discrepâncias nos resultados, como observado nos modelos que utilizam o algoritmo *K-Means*. Porém, notícias que são do tema "ciências" continuaram sendo classificadas como informes. O relatório de classificação, apresentado na Figura 28, mostra métricas que elucidam isso.

Já notícias que discorrem sobre ciências estão sendo classificadas como pertencentes ao tema "informes", como explicitado, isso levou a um baixo *recall*. Os textos que possuem como tema central vagas, assim como nos demais modelos, apresentaram ótimos resultados.

4.1.4 Emissões de Carbono

Com o auxílio da biblioteca "*code carbon*" foi possível estimar as emissões de carbono para os modelos propostos, assim como algumas outra métricas, como é possível observar na Tabela 6.

Figura 28 – Relatório de classificação do modelo XGBoost final para notícias classificadas manualmente.



Fonte: O autor, 2024.

Tabela 6 – Emissões de carbono para os modelos propostos.

Parâmetro	**XGBoost**	**K-Means**	**K-Means com PCA**
Duração (s)	607,97	309,82	250,35
Emissões (kg CO_2)	0,00082012	0,00041693	0,00033652
Taxa de emissões (kg CO_2/seg)	1,34894e-06	1,34572e-06	1,34420e-06
Energia consumida (kWh)	0,00804	0,00409	0,00330
Potência da CPU (W)	42,5	42,5	42,5
Potência da RAM (W)	5,09	5,09	5,09

Fonte: O autor, 2024.

Os dados acima são provenientes do último treinamento realizado dos modelos. Eles mostram o modelo com o algoritmo *XGBoost* emitindo uma maior quantidade de carbono. Entretanto, isso ocorre pela etapa de pré-processamento dos dados dos modelos com *K-Means* não ter sido rastreada, ela se resume à limpeza dos dados e à requisição

do *embedding* da notícia à API da *OpenAI*, que possui um consumo considerável. Desse modo, uma vez que o uso do *hardware* foi o mesmo, o modelo que leva mais tempo para ser processado possui a maior quantidade de emissões de carbono.

4.1.5 Comparação dos Modelos

Conforme exibido, o modelo com o algoritmo *XGBoost*, de forma geral, se mostrou o mais eficiente dentre os três desenvolvidos neste trabalho. O alto desempenho do algoritmo XGBoost, dado o seu comitê de máquinas, contribuiu para este resultado. Também teve um papel importante o preprocessamento dos dados textuais que diferiu daqueles que utilizaram o algoritmo *K-Means*, que fizeram uso da API da *OpenAI* para gerar os *embeddings* dos dados para uso no modelo, enquanto o modelo com *XGBoost* utilizou uma matriz esparsa contendo a frequência de palavras nos documentos. Os *clusters* formados em ambos os modelos com *K-Means* não foram tão precisos, é o que demonstram os resultados obtidos. Isso se deve pela quantidade de dimensões dos *embeddings*. No modelo sem redução de dimensionalidade, permaneceram muitos ruídos na representação numérica do texto. Enquanto aquele que faz uso dessa técnica, reduziu muito a dimensão, perdendo componentes que retiam a maior parte da variância dos dados.

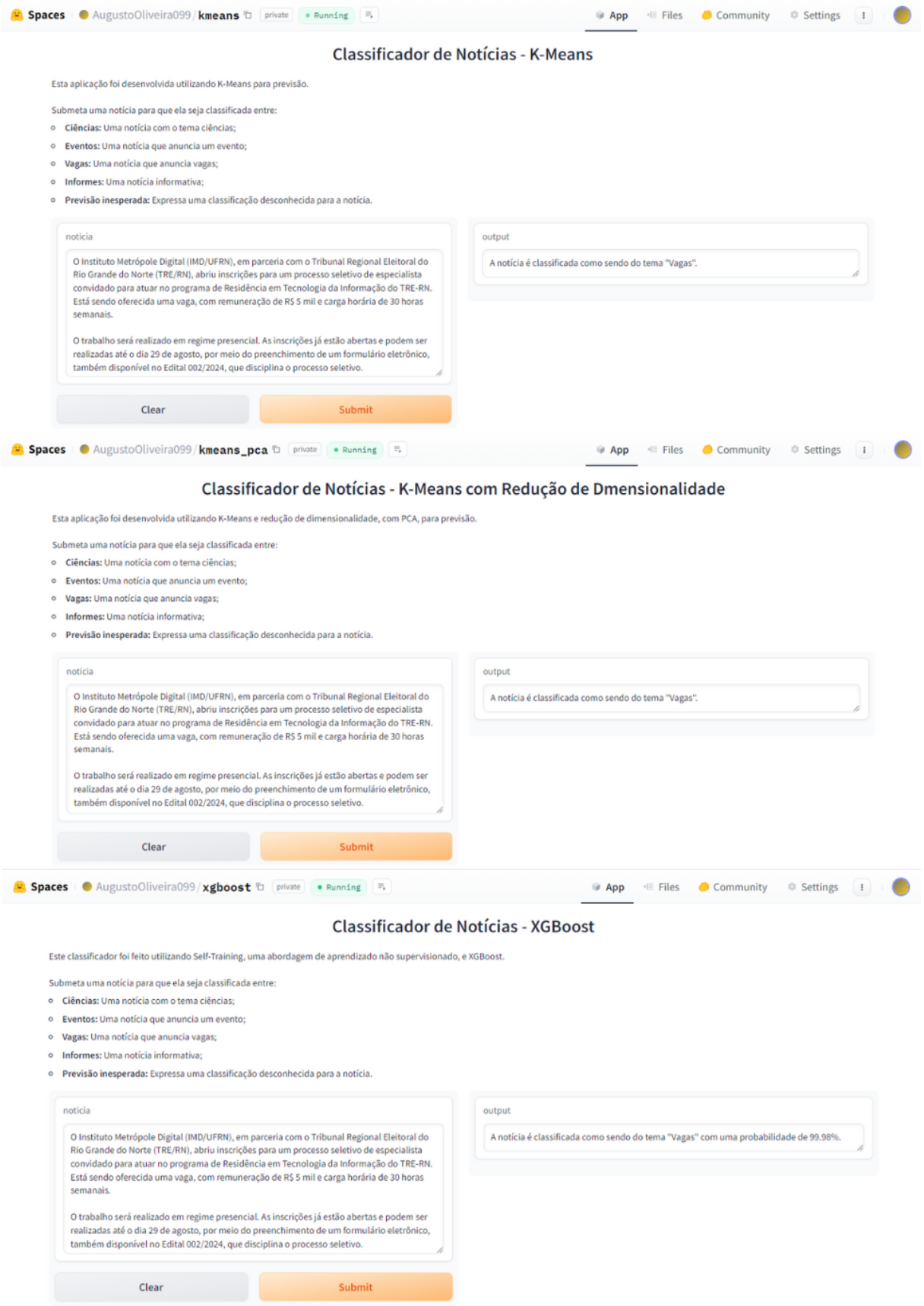
4.2 *Interfaces* do Usuário

As *interfaces* que permitem o usuário interagir com os modelos usam similaridade por cosseno para verificar a semelhança entre o texto de entrada do usuário e os textos armazenados na base de dados. Para tal, foi escolhido um limiar de 85%, ou seja, apenas textos com uma taxa de semelhança menor que essa são armazenados no W&B para um posterior novo treinamento.

Ademais, para armazenar esses textos estava sendo utilizado o DVC, todavia, ele se mostrou lento em produção, ao menos em conjunto com o Google Drive. E para fazer autenticação de forma automática em produção nessa plataforma, é necessário ter uma conta de serviço no Google Cloud Plataform (GCP) e anexar ao código um arquivo com informações privadas. Para o *Hugging Face* - plataforma utiliza para hospedagem das *interfaces* - isso é um problema, uma vez que os usuários podem acessar o código fonte, logo veriam informações confidenciais. O W&B se mostrou mais performático ao salvar grandes conjuntos de dados, como é o caso daqueles que envolvem o modelo *K-Means*, já que os embeddings dos textos são persistidos. Além disso, é mais seguro para o uso neste trabalho, já que necessita apenas de uma chave de acesso, que é facilmente configurada como variável de ambiente privada.

Na Figura 29 estão exemplos de uso das *interfaces* com uma mesma notícia, que é do tema “vagas”.

Figura 29 – Relatório de classificação do modelo XGBoost final para notícias classificadas manualmente.



Fonte: O autor, 2024.

Na Tabela 7 estão os tempos de resposta de cada modelo, que é possível observar na opção de "*logs*" do *Hugging Face*, em cada *interface*, após classificar um texto. O "Tempo 1" da tabela se refere às notícias que não estão armazenadas na base de dados, e que precisam ser salvas no W&B, enquanto "Tempo 2" diz respeito às notícias que estão armazenadas, logo não precisam ser armazenadas.

Tabela 7 – Comparação de tempos para classificar notícias usando diferentes algoritmos.

Algoritmo	**Tempo 1 (s)**	**Tempo 2 (s)**
XGBoost	1,65369	0,07029
K-Means	1,22918	0,50062
K-Means PCA	1,41636	0,30977

Fonte: O autor, 2024.

Primeiro, foi feita a classificação de uma notícia que não estava na base de dados dos modelos. Em seguida, foi feita a classificação da mesma notícia. É possível notar a diferença no tempo de resposta em ambos os casos. Isso gera fluidez e impede que existam textos com informações muito similares na base de dados.

O maior tempo para o modelo *XGBoost* se dá pela atualização da base de dados e pela atualização da matriz de características com a representação numérica da nova entrada. Como são duas requisições para o W&B, isso demanda mais tempo. Em contrapartida, quando é preciso realizar uma classificação de um notícia similar a uma outra da base de dados, ele se mostra mais rápido, pois não necessita fazer requisições assíncronas.

Para os modelos com *K-Means*, todas as classificações geram uma consulta ao modelo da *OpenAI* para gerar o embbeding da notícia, e esse vetor é armazenado no W&B, o que gera um certo tempo de processamento. Quando são notícia similares, apenas a requisição para a API da *OpenAI* é feita. Como explicado na seção 3.1, a pricipal diferença entre os modelos com o algoritmo *K-Means* é a adição da redução de dimensionalidade em um deles. Isso gera um tempo de processamento maior, o que explica a diferença apresentada para novas notícias. Todavia, para o teste feito com a notícia similar, o comportamento se invertou, isso pode ter se dado por uma oscilação na rede, gerando uma demora um pouco maior para o modelo *K-Means* ao fazer a requisição assíncrona do *embedding*, uma vez que os tempos gastos possuem a mesma ordem de grandeza.

5 CONCLUSÃO

A classificação das notícas da Universidade Federal do Rio Grande do Norte (UFRN) entre “ciências”, “eventos”, “vagas” e “informes”, com *K-Means*, *K-Means* com redução de dimensionalidade e *XGBoost* como modelos propostos, mostrou resultados promissores, principalmente com o modelo que utiliza o algoritmo *XGBoost* para previsão, comprovando, assim, sua utilidade prátiva na classificação das notícias da universidade.

Ademais, a análise de custo-benefício, considerando o custo em tokens, dado os novos modelos de *embeddings* da *OpenAI*, se mostrou economicamente viável, uma vez que para processar todas as notícias produzidas até hoje foram gastos U$ 1.448. Esse custo é necessário para os modelos que usam o algoritmo *K-Means*, uma vez que apenas eles fazem uso da API da *OpenAI*. Além disso, para escalar as soluções, é precido aumentar o poder de processamento do *Weigths & Biases* e do *Hugging Face* aderindo a planos pagos.

Vale ressaltar que as três soluções empregaram eficientemente modelos para a classificação de textos no que diz respeito às notícias da UFRN. Elas foram contempladas por uma interface amigável e de fácil utilização criada por meio do *Gradio*, para interação com os modelos, e hospedada no *Hugging Face*.

O modelo que apresentou melhor desempenho, no geral, conforme a acurácia, precisão, *recall* e *F1-score*, foi aquele que utiliza o algoritmo *XGBoost*. Contudo, apesar dos outros modelos terem apresentado uma eficiência razoável, poderiam ter seus desempenhos melhorados caso o número de *features* para a redução de dimensionalidade fosse um valor intermediário entre dois, que foi o escolhido, e o valor total retornado pela API da *OpenAI*, que contém muitos ruídos. Isso é evidenciado com a variância perdida com nesse processo.

Como trabalho futuro, pretende-se treinar um modelo com uma quantidade intermediária de componentes para a redução de dimensionalidade com o intuito de avaliar se isso resulta em um melhor desempenho. Além disso, o classificador de notícias será implantado na UFRN para que a comunidade acadêmica possa utilizá-lo.

REFERÊNCIAS

BIASES, W. . *Weights Biases Documentation*. 2024. Disponível em: https://docs.wandb.ai/. Acesso em: 29 de agosto de 2024.

BIRD, S.; KLEIN, E.; LOPER, E. *Natural language processing with Python: analyzing text with the natural language toolkit*. [S.l.]: "O'Reilly Media, Inc.", 2009.

BISHOP, C. M. Pattern recognition and machine learning. *Springer google schola*, v. 2, p. 1122–1128, 2006.

BOESCH, G. *Ensemble Learning: A Combined Prediction Model (2024 Guide)*. 2024. Disponível em: https://viso.ai/deep-learning/ensemble-learning/. Acesso em: 28 de agosto de 2024.

BOYKIS, V. *What are embeddings?* 2023. Disponível em: <https://github.com/veekaybee-/what_are_embeddings>https://github.com/veekaybee/what_are_embeddings.

BROWNLEE, J. Deep learning for natural language processing. *Machine Learning Mystery, Vermont, Australia*, v. 322, 2017.

CHOWDHARY, K.; CHOWDHARY, K. Natural language processing. *Fundamentals of artificial intelligence*, Springer, p. 603–649, 2020.

DEVELOPERS, S. learn. *Tfidf Term Weighting*. 2024. Disponível em: https://scikit-learn.org/stable/modules/feature_extraction.html#tfidf-term-weighting. Acesso em: 27 de agosto de 2024.

DVC. *Get Started with DVC*. 2024. Disponível em: https://dvc.org/doc/start. Acesso em: 29 de agosto de 2024.

GASPARETTO, A. et al. A survey on text classification algorithms: From text to predictions. *Information*, MDPI, v. 13, n. 2, p. 83, 2022.

GRADIO. *Quickstart*. 2024. Disponível em: https://www.gradio.app/guides/quickstart. Acesso em: 29 de agosto de 2024.

GREFENSTETTE, G. Tokenization. In: *Syntactic wordclass tagging*. [S.l.]: Springer, 1999. p. 117–133.

IKONOMAKIS, M. et al. Text classification using machine learning techniques. *WSEAS transactions on computers*, Citeseer, v. 4, n. 8, p. 966–974, 2005.

KURITA, T. Principal component analysis (pca). *Computer vision: a reference guide*, Springer, p. 1–4, 2019.

LI, B.; HAN, L. Distance weighted cosine similarity measure for text classification. In: SPRINGER. *Intelligent Data Engineering and Automated Learning–IDEAL 2013: 14th International Conference, IDEAL 2013, Hefei, China, October 20-23, 2013. Proceedings 14*. [S.l.], 2013. p. 611–618.

LI, Z. et al. Social media rumor refuter feature analysis and crowd identification based on xgboost and nlp. *Applied Sciences*, MDPI, v. 10, n. 14, p. 4711, 2020.

OPENAI. *About*. 2024. Disponível em: https://openai.com/about/. Acesso em: 29 de agosto de 2024.

OPENAI. *New embedding models and API updates*. 2024. Disponível em: https://openai.com/index/new-embedding-models-and-api-updates/. Acesso em: 29 de agosto de 2024.

PEREIRA, L. C. *Introdução ao Hugging Face: Primeiros Passos em Inteligência Artificial*. 2024. Disponível em: https://hub.asimov.academy/tutorial/introducao-ao-hugging-face-primeiros-passos-em-inteligencia-artificial/. Acesso em: 29 de agosto de 2024.

PLISSON, J. et al. A rule based approach to word lemmatization. In: CITESEER. *Proceedings of IS*. [S.l.], 2004. v. 3, p. 83–86.

SAMEK, W.; MÜLLER, K.-R. Towards explainable artificial intelligence. *Explainable AI: interpreting, explaining and visualizing deep learning*, Springer, p. 5–22, 2019.

SILVA, F. d. C. T. Estudos comparados como método de pesquisa: a escrita de uma história curricular por documentos curriculares. *Revista Brasileira de Educação*, SciELO Brasil, v. 21, p. 209–224, 2016.

SINAGA, K. P.; YANG, M.-S. Unsupervised k-means clustering algorithm. *IEEE access*, IEEE, v. 8, p. 80716–80727, 2020.

SORZANO, C. O. S.; VARGAS, J.; MONTANO, A. P. A survey of dimensionality reduction techniques. *arXiv preprint arXiv:1403.2877*, 2014.

TECUCI, G. Artificial intelligence. *WIREs Comput. Stat.*, John Wiley & Sons, Inc., USA, v. 4, n. 2, p. 168–180, feb 2012. ISSN 1939-5108. Disponível em: <https://doi.org/10.1002/wics.200>https://doi.org/10.1002/wics.200.

TRIPATHI, M. *How to process textual data using TF-IDF in Python*. 2018. Disponível em: https://www.freecodecamp.org/news/how-to-process-textual-data-using-tf-idf-in-python-cd2bbc0a94a3. Acesso em: 27 de agosto de 2024.

WAZLAWICK, R. S. *Metodologia de pesquisa para ciência da computação*. [S.l.]: Elsevier Rio de Janeiro, 2009. v. 2.